Graphics & Visualization

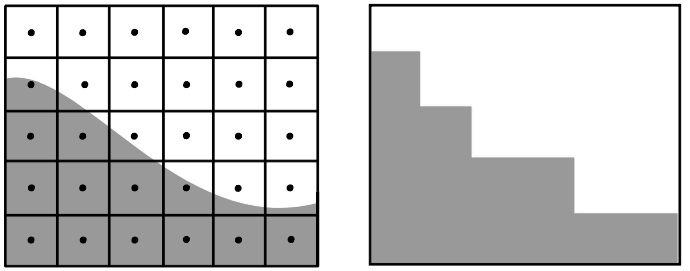
Chapter 2

Rasterization Algorithms

Rasterization

- 2D display devices consist of discrete grid of pixels

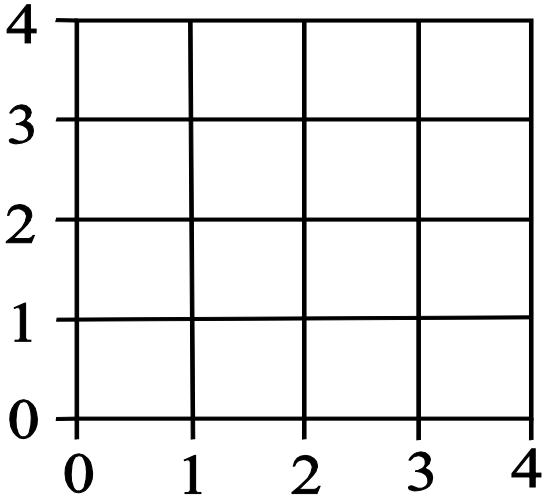
- **Rasterization:** converting 2D primitives into a discrete pixel representation
  - Rastarization is a *sampling* process
  - Rasterization is prone to aliasing if sampling frequency below Nyquist

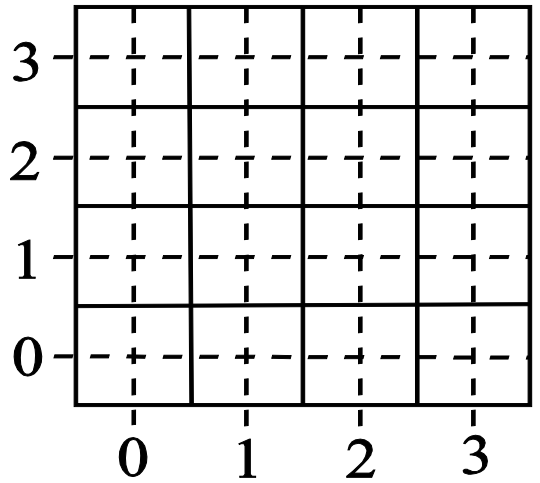


Sampling Reconstruction

Rasterization (2)

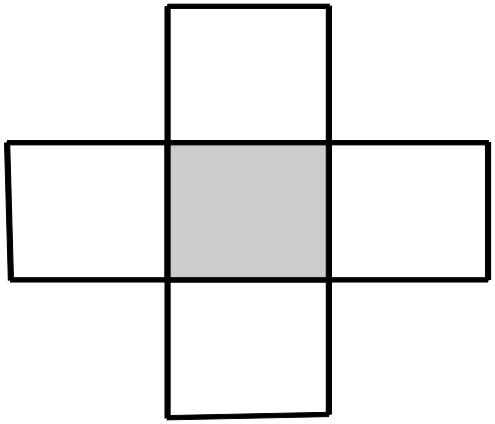
- The complexity of rasterization is O(Pp), where P is the number of primitives and p is the number of pixels
  - Previous stages in the pipeline have complexity O(Pv) where v: vertices per primitive
  - As $p>>v$ rasterization must be very efficient to avoid bottleneck
- There are 2 main ways of viewing the grid of pixels:
  - Half – Integer Centers
  - Integer Centers (shall be used)



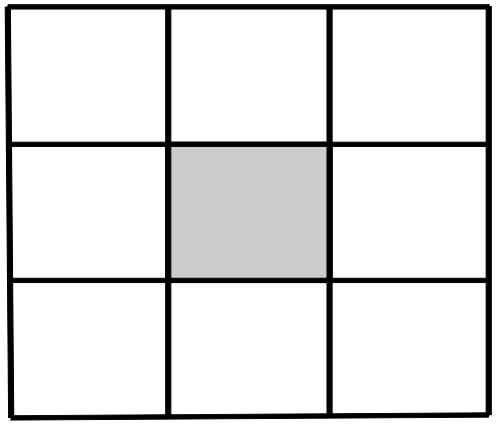


Rasterization (3)

- **Connectedness:** which are the neighbors of a pixel?
  - 4 – connectedness
  - 8 – connectedness



4 – Connectedness



8 - Connectedness

- Challenges in designing a rasterization algorithm:
  - Determine the pixels that accuracy describe the primitive
  - Efficiency

Mathematical Curves

- Curves on the plane can be defined using
  - **Algebraic Representation:**
    - Explicit $y=f(x)$
    - Implicit $f(x,y)=0$
  - **Parametric Representation** $f(t)=(x(t),y(t))$

  - Most useful to us are the Implicit Algebraic and the Parametric

Mathematical Curves (2)

- **Implicit Algebraic Form:**

e.g.:

$$f(x,y)\begin{cases} < 0, & \text{implies point(x,y) is 'inside' the curve} \\ = 0, & \text{implies point(x,y) is on the curve} \\ > 0, & \text{implies point(x,y) is 'outside' the curve} \end{cases}$$

- **Parametric Form:**
  - Function of a parameter $t \in [0, 1]$
  - **t** corresponds to arc length along the curve
  - The curve is traced as *t* goes from 0 to 1

e.g.: $l(t) = (x(t), y(t))$

Mathematical Curves (3)

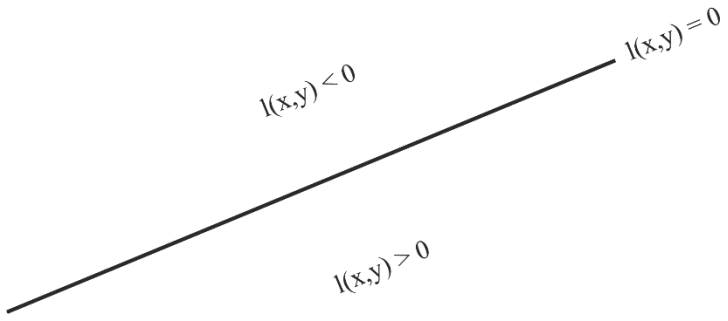
- **Implicit Algebraic Form:**
  - **line:** $l(x, y) \equiv ax + by + c = 0$

    where *a, b, c* : line coefficients derived from endpoints

    if $l(x, y) = 0$ then point (x, y) is on the curve

    else if $l(x, y) < 0$ then point (x, y) is on one half-plane

    else if $l(x, y) > 0$ then point (x, y) is on the other half-plane



Mathematical Curves (4)

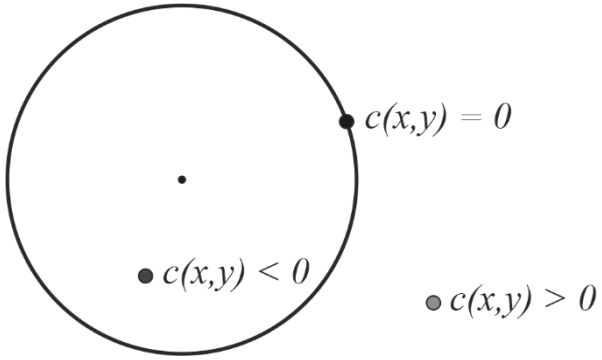
- **Implicit Algebraic Form:**
  - **circle:** $c(x, y) \equiv (x - x_c)^2 + (y - y_c)^2 - r^2 = 0$

    where (x_c, y_c) : the center of the circle & r: circle's radius

    if $c(x, y) = 0$ then point (x, y) is on the circle

    else if $c(x, y) < 0$ then point (x, y) is inside the circle

    else if $c(x, y) > 0$ then point (x, y) is outside the circle



Mathematical Curves (5)

- **Parametric Form:**
  - **line** from $\mathbf{p_1}(x_1, y_1)$ to $\mathbf{p_2}(x_2, y_2)$:

    $\mathbf{l}(t) = (x(t), y(t))$

    where $x(t) = x_1 + t\,(x_2 - x_1)$,

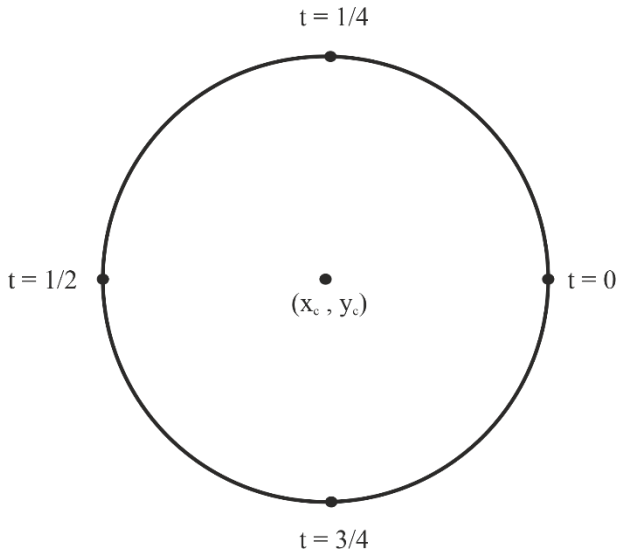
    $y(t) = y_1 + t\,(y_2 - y_1)$,

    $t \in [0,1]$
  - **circle:** $\mathbf{c}(t) = (x(t), y(t))$

    where $x(t) = x_c + r\ \cos(2\pi t)$,

    $y(t) = y_c + r\ \sin(2\pi t)$,

    $t \in [0,1]$



Finite Differences

- Functions that define primitives need to be evaluated on the pixel grid for each pixel ⇨ wasteful
- Cut this cost by taking advantage of finite differences
- Forward differences (fd):
  - First (fd) : $\delta f_i = f_{i+1} - f_i$
  - Second (fd): $\delta^2 f_i = \delta f_{i+1} - \delta f_i$
  - k^{th} (fd): $\delta^k f_i = \delta^{k-1} f_{i+1} - \delta^{k-1} f_i$
- Implicit functions can be used to decide if the pixel belongs to the primitive

  e.g.: pixel(x, y) is included if $|f(x, y)| < e$,

  where e: related to the line width

Finite Differences (2)

- Examples:
  - Evaluation of the **line** function incrementally:

→from pixel *(x, y)* to pixel *(x+1, y)*

Calculation of the forward differences of the implicit line equation in the x direction from pixel x to pixel $x+1$:

$$\delta_x l(x,y) = l(x+1,y) - l(x,y) = a$$

Compute $l(x,y) + \delta_x l(x,y) = l(x,y) + a$

→from pixel *(x, y)* to pixel *(x, y+1)*

Calculation of the forward differences of the implicit line equation in the y direction from pixel y to pixel $y+1$:

$$\delta_y l(x,y) = l(x,y+1) - l(x,y) = b$$

Compute $l(x,y) + \delta_y l(x,y) = l(x,y) + b$

Finite Differences (3)

- Examples:
  - Evaluation of the **circle** function incrementally:

→from pixel *(x, y)* to pixel *(x+1, y)*

Calculation of the forward differences of the implicit circle equation. Since it has degree 2 there are two forward differences in the x direction from pixel x to pixel $x+1$:

$$\delta_x c(x,y) = c(x+1,y) - c(x,y) = 2(x - x_c) + 1$$

$$\delta_x^2 c(x,y) = \delta_x c(x+1,y) - \delta_x c(x,y) = 2$$

Compute $\delta_x c(x,y) = \delta_x c(x-1,y) + \delta_x^2 c(x,y)$

$$c(x+1,y) = c(x,y) + \delta_x c(x,y)$$

→from pixel *(x, y)* to pixel *(x, y+1)*: similar by adding $\delta_y c(x,y)$ and $\delta_y^2 c(x,y)$

Rasterizing Primitives

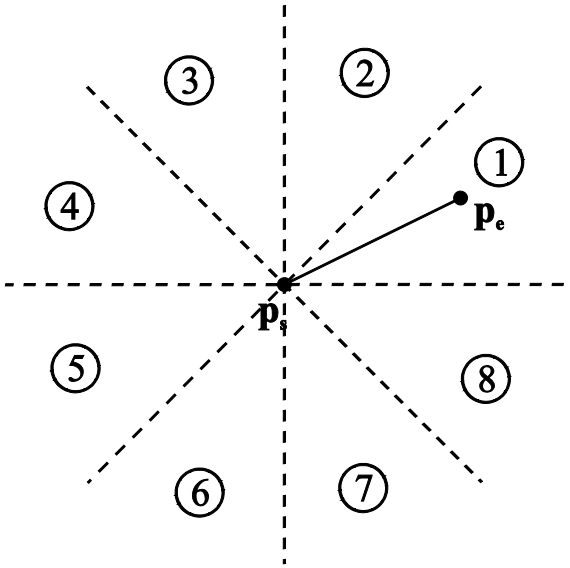
- One way is to evaluate functions that define a primitive over the pixel grid and test for pixel inclusion
  - For example the distance of a pixel *(x,y)* from a line defined by the implicit algebraic function is $\frac{|l(x,y)|}{\sqrt{a^2+b^2}}$
  - However, even incremental evaluation can be costly
  - Methods that *track* primitives can be more efficient

Point Rasterization

- Vertex only models (point clouds) are becoming more popular
  - Natural output of 3D scanners
  - Points can be represented as squares
  - Pixel centers are tested for inclusion in such a square
  - Antialiased versions exist

Line Rasterization

- Desired qualities of a line rasterization algorithm:
  - Selection of the nearest pixels to the mathematical path of the line
  - Constant line width, independent of the slope of the line
  - No gaps
  - High efficiency



The 8 octants with an example line in the first octant

Line Rasterization Algorithm 1

- Draw a line from pixel $\mathbf{p_s} = (x_s, y_s)$ to pixel $\mathbf{p_e} = (x_e, y_e)$ in the first octant

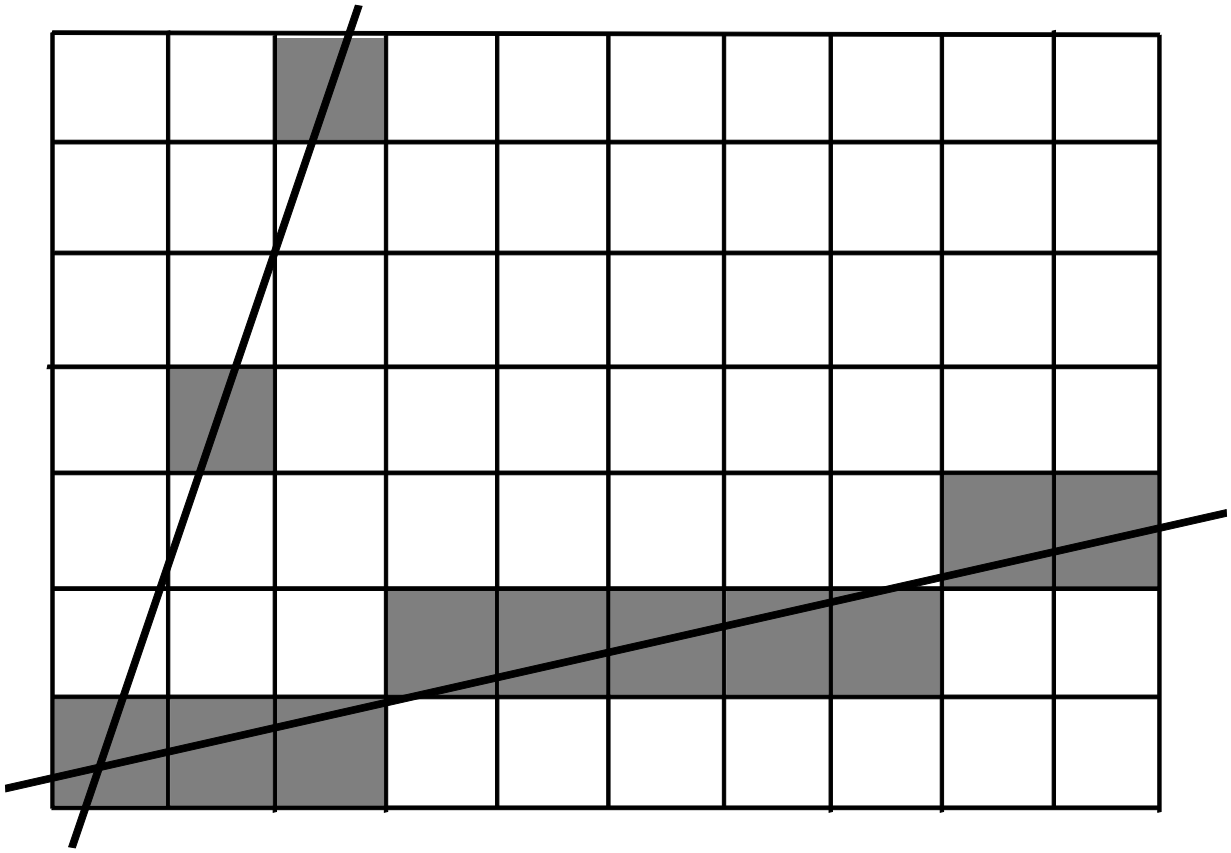
- Slope of the line: $s = \frac{y_e - y_s}{x_e - x_s}, \quad y = y_s + round(s \cdot (x - x_s)), \quad x = x_s, \ldots, x_e$

Algorithm:

```
line1 ( int xs, int ys, int xe, int ye, colour c )  {
        float s; int x, y;
        s = (ye - ys) / (xe - xs);  (x, y) = (xs, ys);
        while (x <= xe) {
                setpixel (x, y, c);
                x = x + 1;
                y = ys + round(s * (x - xs));
        }
}
```

Line Rasterization Algorithm 1 (2)

- Using `line1` algorithm in the first and second octants:



Line Rasterization Algorithm 2

- Avoid rounding operation by splitting y value into an integer and a float part e
- Compute its value incrementally

Algorithm:

```
line2 ( int xs, int ys, int xe, int ye, colour c )  {
    float s, e;  int x, y;
    e = 0;     s = (ye - ys) / (xe - xs);     (x, y) = (xs, ys);
    while (x <= xe) {
        /* assert -1/2 <= e < 1/2 */
        setpixel(x, y, c);
        x = x + 1;
        e = e + s;
        if (e >= 1/2) {
            y = y + 1;
            e = e - 1;
        }
    }
}
```

Line Rasterization Algorithm 2 (2)

- Algorithm `line2` resembles the leap year calculation
  - The slope is added to the *e* variable at each iteration until it makes up more than half a unit & then the line leaps up by 1.
  - The integer *y* variable is incremented and *e* is correspondingly reduced, so that the sum of the 2 variables is unchanged.
  - Similarly, the year has approximately 365,25 days but calendars are designed with an integer number of days.
  - We add a day every 4 years to make up for the error being accumulated.

Bresenham Line Algorithm

- Replace the floating point variables in `line2` by integers
- Multiplying the leap decision variables by $dx = x_e - x_s$ makes s and e integers
- The leap decision becomes $e \geq \left\lfloor \frac{dx}{2} \right\rfloor$ because e is integer
- $\left\lfloor \frac{dx}{2} \right\rfloor$ can be computed by a numerical shift
- For more efficiency replace the test $e \geq \left\lfloor \frac{dx}{2} \right\rfloor$ by $e \geq 0$ using an initial subtraction of $\left\lfloor \frac{dx}{2} \right\rfloor$ from e

Bresenham Line Algorithm (2)

- Floating point variables are replaced by integers

Bresenham's Algorithm

```
line3 ( int xs, int ys, int xe, int ye, colour c )  {
    int x, y, e, dx, dy;
    dx = (xe - xs); dy=(ye - ys); e = - (dx >> 1); (x, y)=(xs, ys);
    while (x <= xe) {
        /* assert -dx <= e < 0 */
        setpixel(x, y, c);
        x = x + 1;
        e = e + dy;
        if (e >= 0) {
            y = y + 1;
            e = e - dx;
        }
    }
}
```

Bresenham Line Algorithm (3)

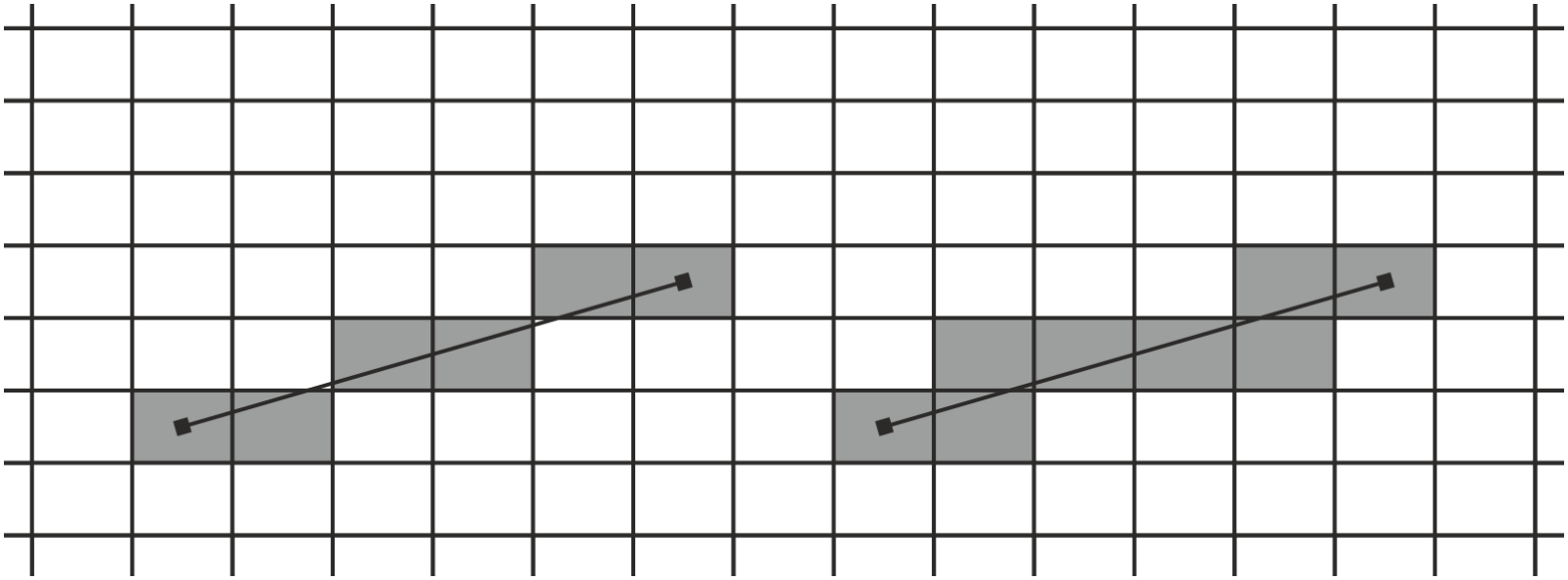
- Suitable for lines in the first octant
- Changes for other octants according to the following table
- Two versions can handle all octants: x major and y major

Octant	Major Axis	Minor Axis Variable
1	x	increasing
2	y	increasing
3	y	decreasing
4	x	increasing
5	x	decreasing
6	y	decreasing
7	y	increasing
8	x	decreasing

- Meets the requirements of a good line rasterization algorithm

Grid Traversal

- Determine sequence of grid cells traversed by a line
  - Useful e.g. in ray tracing
- All grid cells must be determined, as opposed to line rasterization



Line rasterization VS Grid traversal

Grid Traversal (2)

- Consider a ray and a 2D grid (extension to 3D is trivial)
  - Ray from **s**=*(x,y)* in direction $\vec{v}$: parametric representation **s**+$t\vec{v}$
  - *tNextx / tNexty*: parametric values at next vertical/horizontal cell crossing
  - *tDX / tDY*: parameteric increment for width/height of one cell

```
While /* not termination condition */ {
    select_cell(x,y);
    if (tNextx < tNexty) {
            tNextx += tDX;
            x += stepx;  /* +1 or -1 depending if x inc or dec */
    } else {
            tNexty += tDY;
            y += stepy; /* +1 or -1 */
    }
}
```

Point in Polygon Tests

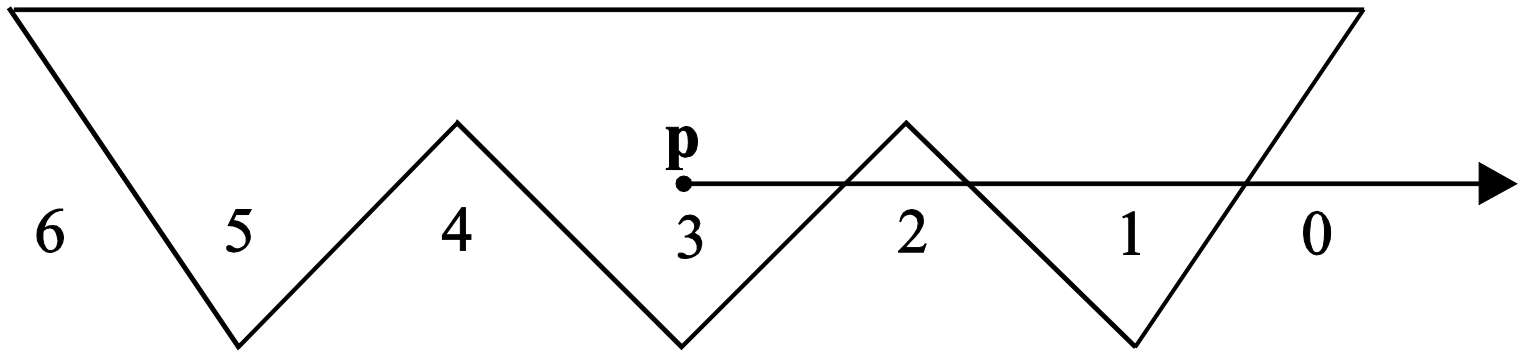
- Polygon: n vertices $(v_0, \ldots, v_{n-1})$ and n edges form a closed curve $v_0, v_1, \ldots, v_{n-1}, v_0$

- **Jordan Curve Theorem:** A continuous simple closed curve in the plane separates the plane into 2 regions. The *inside* and the *outside*

- For efficient rasterization we need to know if a pixel **p** is inside a polygon *P*. There are two types of inclusion tests:
  - Parity test
  - Winding number

Point in Polygon Tests (2)

- Parity Test:
  - Draw a half line from pixel p in any direction
  - Count the number of intersections of the line with the polygon *P*
  - If #intersections == odd number then **p** is inside *P*
  - Otherwise **p** is outside *P*

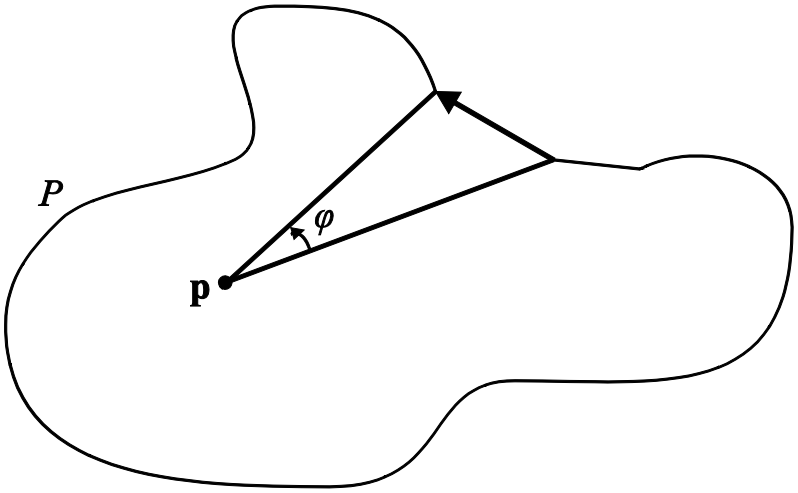


Point in Polygon Tests (3)

- Winding Number Test:
  - $\omega(P, \mathbf{p})$ counts the # of revolutions completed by a ray from **p** tracing P

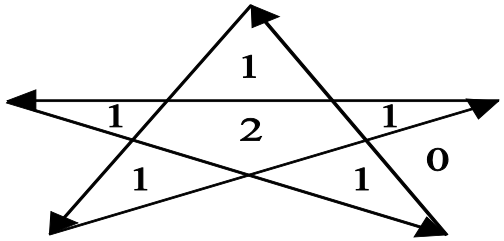
$$\omega(P,\mathbf{p}) = \frac{1}{2\pi}\int d\varphi$$

  - For every counterclockwise revolution $\omega(P, \mathbf{p})$ ++
  - For every clockwise revolution $\omega(P, \mathbf{p})$--
  - If $\omega(P, \mathbf{p})$ is odd then **p** is inside P
  - Otherwise **p** is outside P

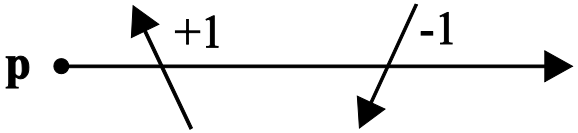


Point in Polygon Tests (4)

- The winding number test for point in polygon:

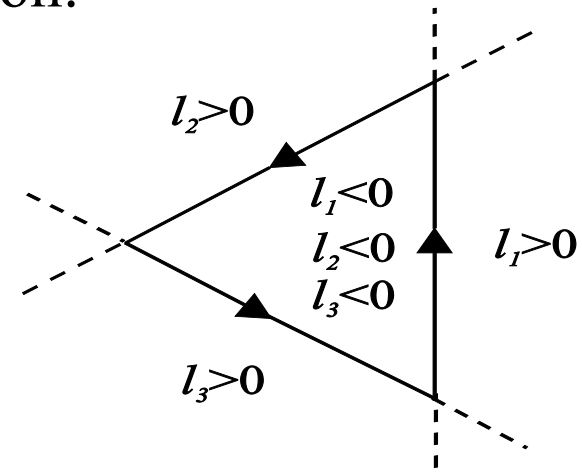


- Simple computation of the winding number:



- The sign test for point in convex polygon:

$$\text{sign}(l_o(\mathbf{p})) = \text{sign}(l_1(\mathbf{p})) = \ldots = \text{sign}(l_{n-1}(\mathbf{p}))$$

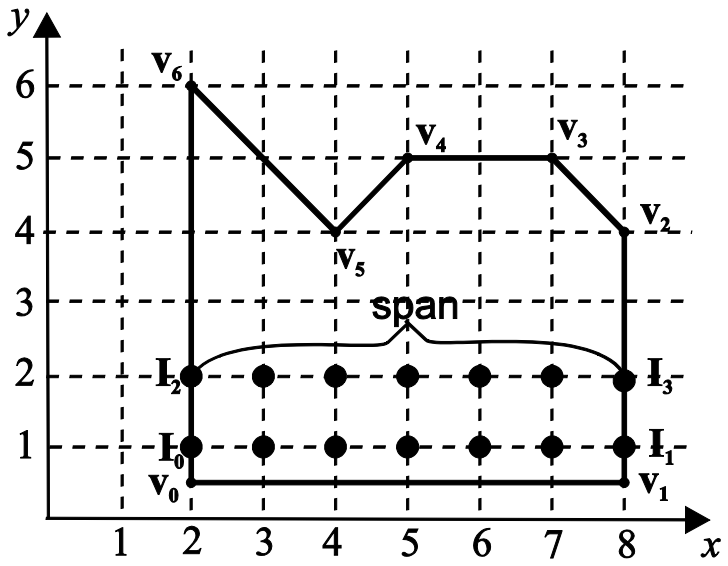


Polygon Rasterization

- Why Polygon Rasterization ?
  - All our primitives end up in 2D screen and need to be drawn
    - Real-time 3D graphics usually only uses triangles
    - General polygons can be triangulated
  - General polygons is useful in 2D drawing and is instructive
  - Area filling is also useful in 2D drawing and other applications

Polygon Rasterization

- Basic Polygon Rasterization Algorithm:
  - Based on the parity test
  - Steps:
    1. Compute intersections **I**(x, y) of every edge with all the scanlines it intersects & store them in a list
    2. Sort the intersections by (y, x)
    3. Extract spans from the list & set the pixels between them



Singularities

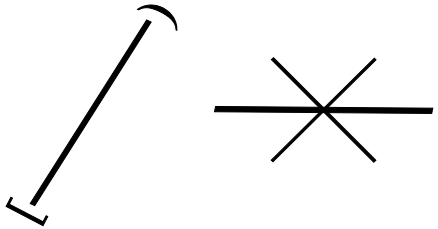
- Basic Polygon Rasterization Algorithm:
  - inefficient due to the cost of intersection computations

- Problem:
  - if a polygon vertex falls exactly on a scanline:
    count 2, 1 or 0 intersections ?

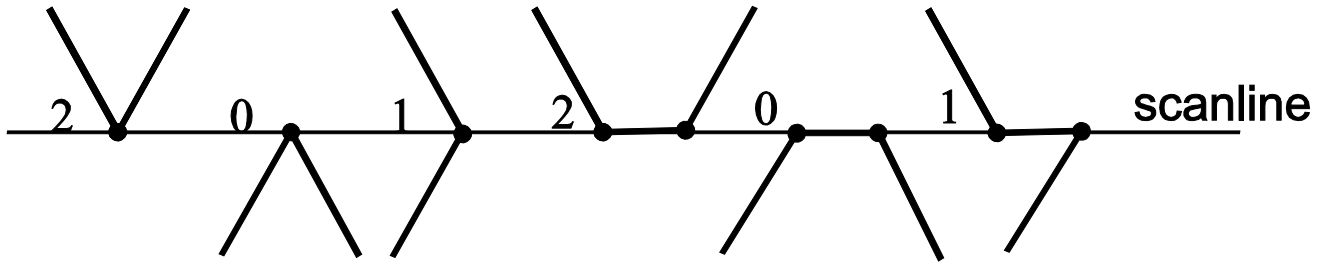
- Solutions:
  - regard edge as closed on the vertex with min y and open on the vertex with max y
  - ignore horizontal edges

Singularities (2)

- Rule for Treating Intersection Singularities



- Effect of Singularities Rule on Singularities



Scanline Polygon Rasterization Algorithm

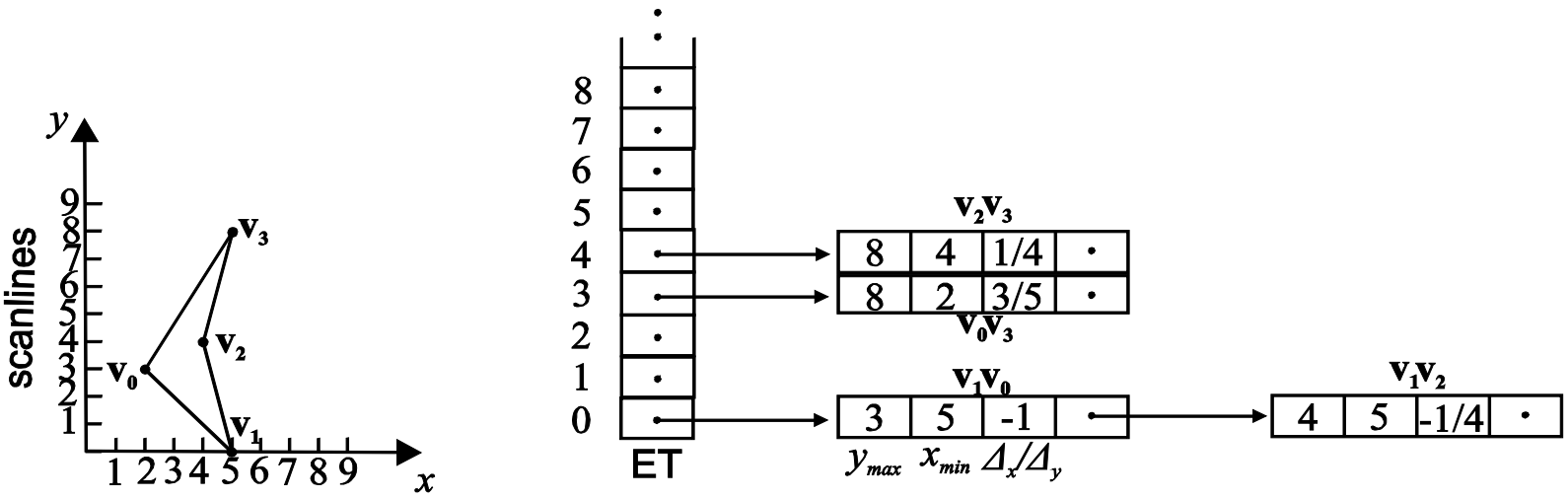
- Takes advantage of scanline coherence & edge coherence
- Uses an Edge Table (ET) and an Active Edge Table (AET)

Algorithm:

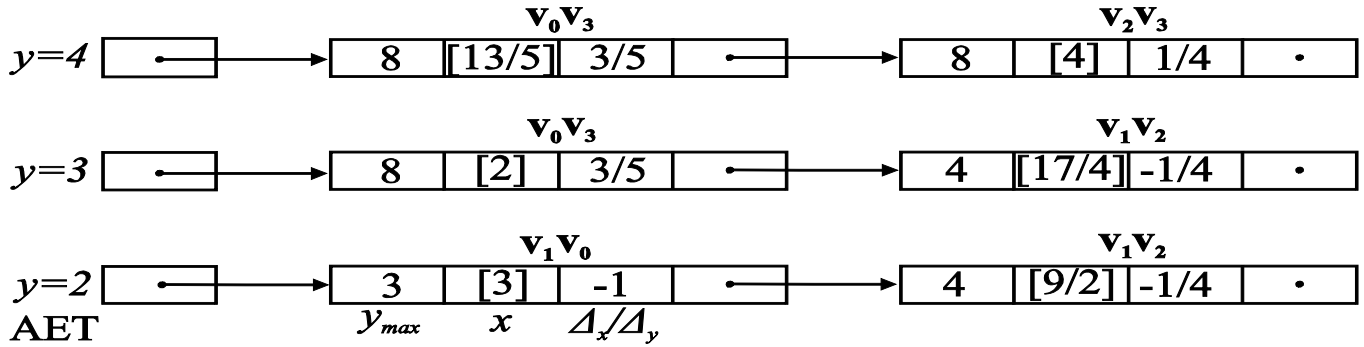
1. Construct the polygon ET, containing the maximum y, the min x and the inverse slope of each edge (y_{max}, x_{min}, 1/s). The record of an edge is inserted in the bucket of its minimum y coordinate.
2. For every scanline y that intersects the polygon in an upward sweep:
   (a) Update the AET edge intersections for the current scanline:
   $x = x + 1/s$.
   (b) Insert edges from y bucket of ET into AET.
   (c) Remove edges from AET whose $y_{max} \leqslant y$.
   (d) Re-sort AET on x.
   (e) Extract spans from the AET and set their pixels.

Scanline Polygon Rasterization Algorithm (2)

- A polygon and its Edge Table (ET)

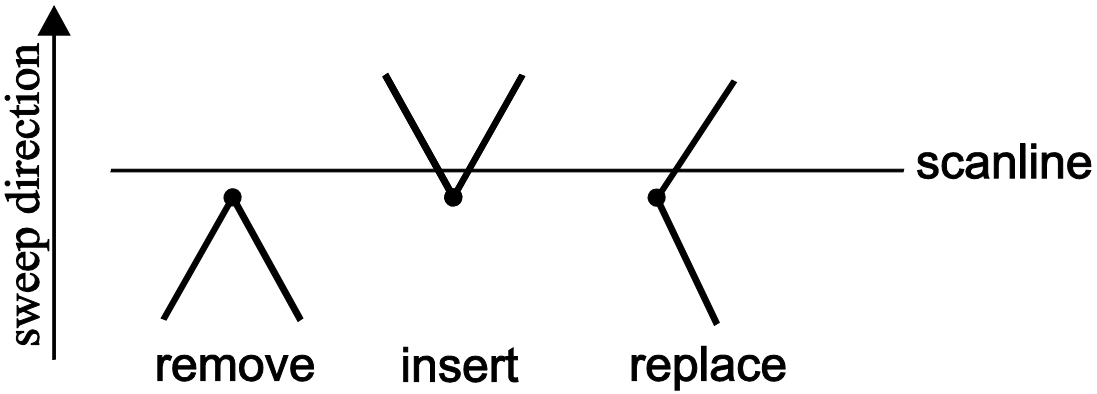


- Example states of the AET



Scanline Polygon Rasterization Algorithm (3)

- The edges that populate the AET change at polygon vertices according to the following figure:

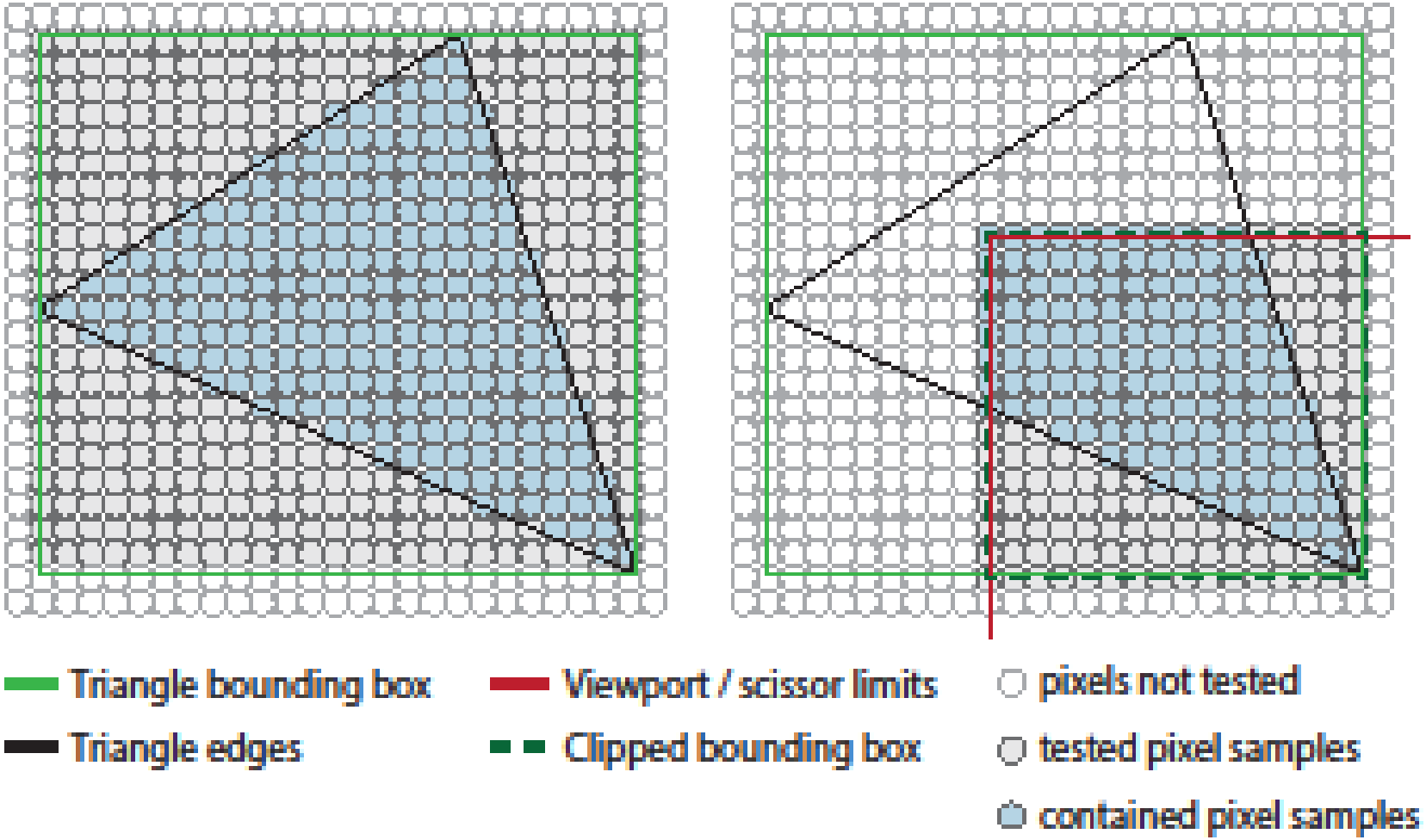


Updating the AET

Triangle Rasterization

- **Triangle**: simplest, planar, convex polygon
  - Triangulation algorithms
- Determine the pixels covered by a triangle → perform an inside test on all the pixels **q** of the triangle's scissored bounding box
  - Incrementally evaluate 3 edge functions
  - Check for same sign
  - Vertex attribute interpolation is also performed

Triangle Rasterization (2)



Triangle Rasterization (3)

- For 2 vectors $\mathbf{v}_1 = [x_1, y_1]^T$ and $\mathbf{v}_2 = [x_2, y_2]^T$ the **signed** area of the formed triangle is:

$$A_t = \frac{1}{2}\begin{vmatrix} x_1 & x_2 \\ y_1 & y_2 \end{vmatrix}$$

- Taking a triangle edge $\mathbf{p_0 p_1}$ and an arbitrary point $\mathbf{q}$ we can define $\mathbf{v}_1 = \mathbf{p_1} - \mathbf{p}_0$ and $\mathbf{v}_2 = \mathbf{q} - \mathbf{p}_0$ and thus the discriminant of $\mathbf{q}$ with respect to the triangle edge is:

$$F_{01}(\mathbf{q}) = \begin{vmatrix} x_1 - x_0 & x_q - x_0 \\ y_1 - y_0 & y_q - y_0 \end{vmatrix}$$

$$= (y_1 - y_0)x_q + (x_1 - x_0)y_q + (x_0 y_1 + y_0 x_1)$$
$$= A_{01}x_q + B_{01}y_q + C_{01}$$

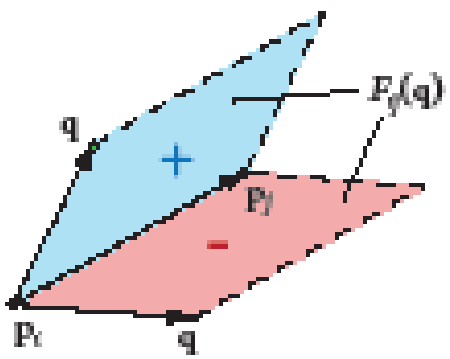
- If +ve then $\mathbf{q}$ is on the +ve halfplane of the edge, else on -ve

Triangle Rasterization (4)

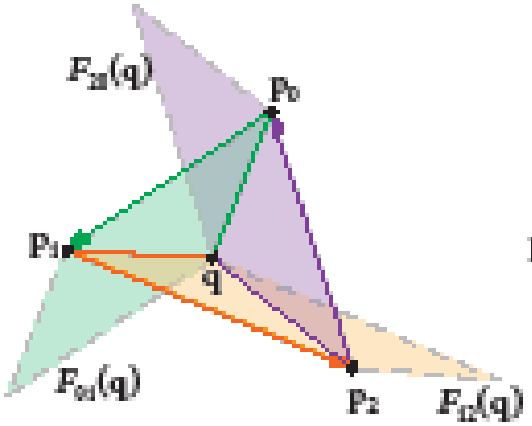
- Similarly for the other 2 triangle edges and **q**:

$$F_{12}(\mathbf{q}) = (y_2 - y_1)x_q + (x_2 - x_1)y_q + (x_1y_2 + y_1x_2)$$
$$= A_{12}x_q + B_{12}y_q + C_{12}$$

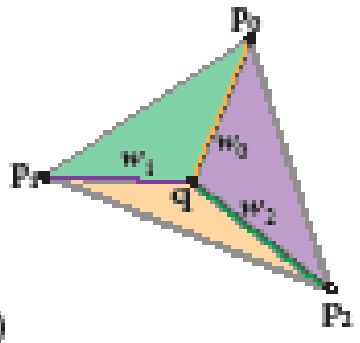
$$F_{20}(\mathbf{q}) = (y_0 - y_2)x_q + (x_0 - x_2)y_q + (x_2y_0 + y_2x_0)$$
$$= A_{20}x_q + B_{20}y_q + C_{20}$$



Edge equation



Edge equations for point in triangle



Barycentric coordinates from edge equations

Triangle Rasterization (5)

- Apart from inclusion in triangle, the edge function values lead to barycentric coordinates for attribute interpolation at **q**:

$$b_0 = F_{12}(\mathbf{q})/a$$
$$b_1 = F_{20}(\mathbf{q})/a$$
$$b_2 = F_{01}(\mathbf{q})/a$$
$$a = F_{12}(\mathbf{q}) + F_{20}(\mathbf{q}) + F_{01}(\mathbf{q})$$

  - The A_{ij}, B_{ij}, C_{ij} can be computed once at triangle setup

- Incremental evaluation of the F_{ij} from (x_q, y_q) to (x_q+1, y_q) is possible with forward differences:

$$F_{ij}(x_q + 1, y_q) = A_{ij}(x_q + 1) + B_{ij}y_q + C_{ij} = F_{ij}(x_q, y_q) + A_{ij}$$

- Evaluation at arbitrary offset (s,t) from an initial evaluation at **c** also easy (more useful for parallelism):

$$F_{ij}(x_c + s, y_c + t) = F_{ij}(x_c, y_c) + A_{ij}s + B_{ij}t$$

Area Filling Algorithms

- A simple approach is **flood fill**

Algorithm:

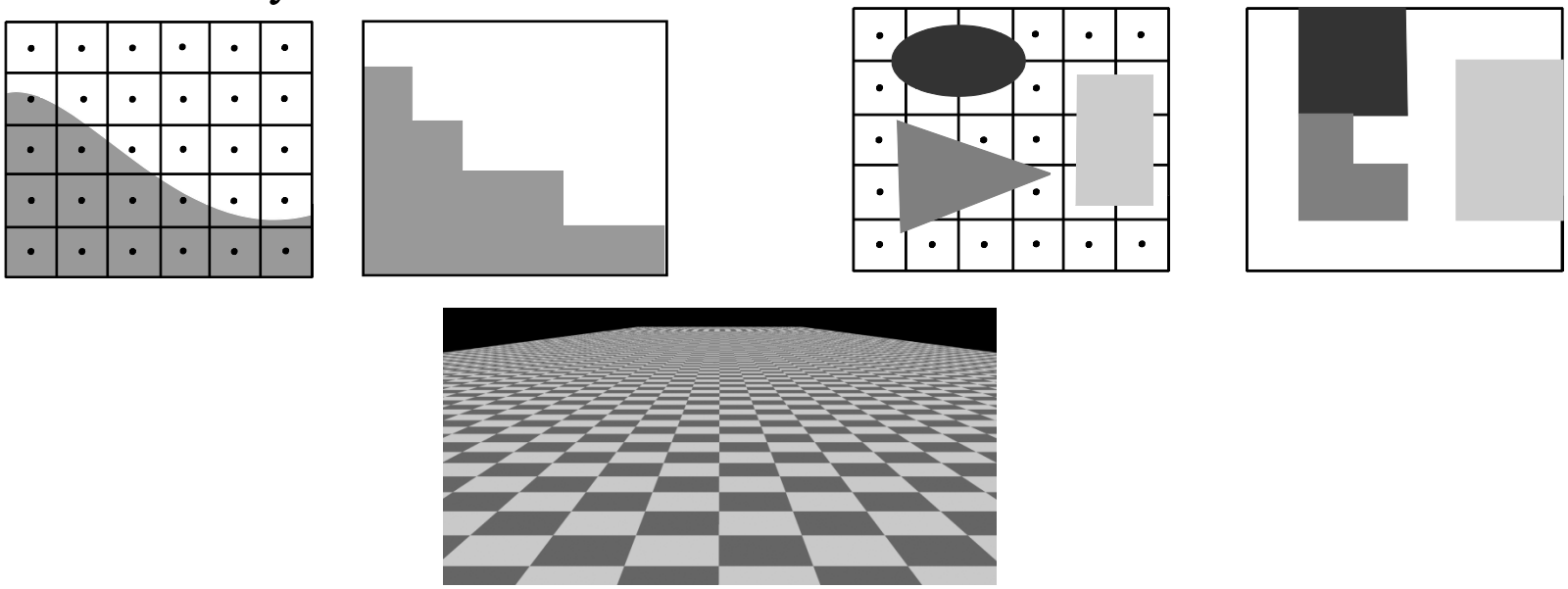
```
flood_fill ( polygon P, colour c ) {
point s;
draw_perimeter ( P, c );
s = get_seed_point ( P );
flood_fill_recur ( s, c );
}

flood_fill_recur ( point (x,y), colour fill_colour ); {
colour c;
c = getpixel(x,y); /* read current pixel colour */
if (c != fill_colour) {
    setpixel(x,y,fill_colour);
    flood_fill_recur((x+1,y), fill_colour ); flood_fill_recur((x-1,y),fill_colour );
    flood_fill_recur((x,y+1), fill_colour ); flood_fill_recur((x,y-1), fill_colour );
    }
}
```

Area Filling Algorithms (2)

- For 4 – connected areas the above 4 recursive calls are sufficient
- For 8 – connected areas 4 extra recursive calls must be added
  - `flood_fill_recur((x+1,y+1), fill_colour );`
  - `flood_fill_recur((x+1,y-1), fill_colour );`
  - `flood_fill_recur((x-1,y+1), fill_colour );`
  - `flood_fill_recur((x-1,y-1), fill_colour );`
- Basic problem its innefficiency

Spatial Anti-aliasing

- Primitive rasterization algorithms sample at pixel centre
- Pixels are **not** mathematical points but have a small area → aliasing effects
- Aliasing effects:
  - jagged appearance of object silhouettes
  - improperly rasterized small objects
  - incorrectly rasterized detail



Anti-aliasing Techniques

- Aliasing occurs because the sampling frequency is below Nyquist
  - Sampling frequency is the pixel resolution, signal is the image
- Anti-aliasing trades intensity resolution to gain spatial resolution

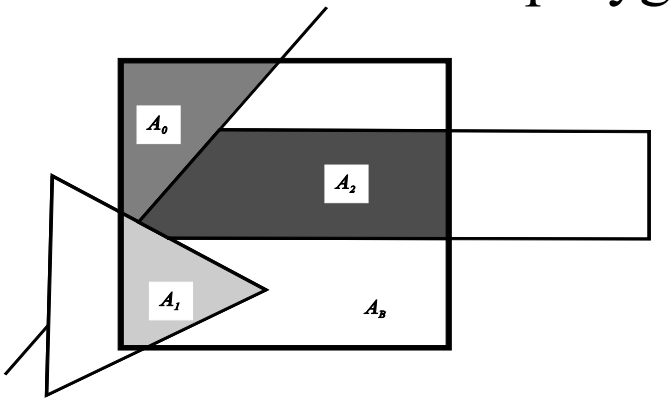
3 categories of anti-aliasing techniques:

- **Pre-filtering**:
  - extract high frequencies before sampling
  - compute % contribution of each primitive within pixel area
- **Post-filtering**:
  - extract high frequencies after sampling
  - increase sampling frequency and averaged down
- **Post-processing**:
  - create display-resolution image and remove aliasing effects

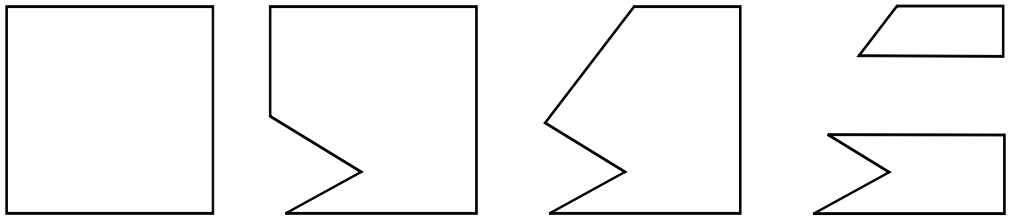
Pre-filtering Anti-aliasing: Catmull

Anti-aliased Polygon Rasterization: **Catmull's Algorithm**

- Consider each pixel as a square window
- Clip all overlapping polygons
- Estimate the visible area of each polygon as a % of the pixel



- A general polygon clipping algorithm is needed, such as Greiner-Horman



Pre-filtering Anti-aliasing: Catmull (2)

Algorithm:

1. Clip all polygons against the pixel window →

   $P_0 \ldots P_{n-1}$: the surviving polygon pieces

2. Eliminate hidden surfaces:

   (a) order by depth polygons $P_0 \ldots P_{n-1}$

   (b) clip against the area formed by subtracting the polygons from the (remaining) pixel window in depth order →

   $P_0 \ldots P_{m-1}$ ($m \leqslant n$) the visible parts of polygons &

   $A_0 \ldots A_{m-1}$ their respective areas

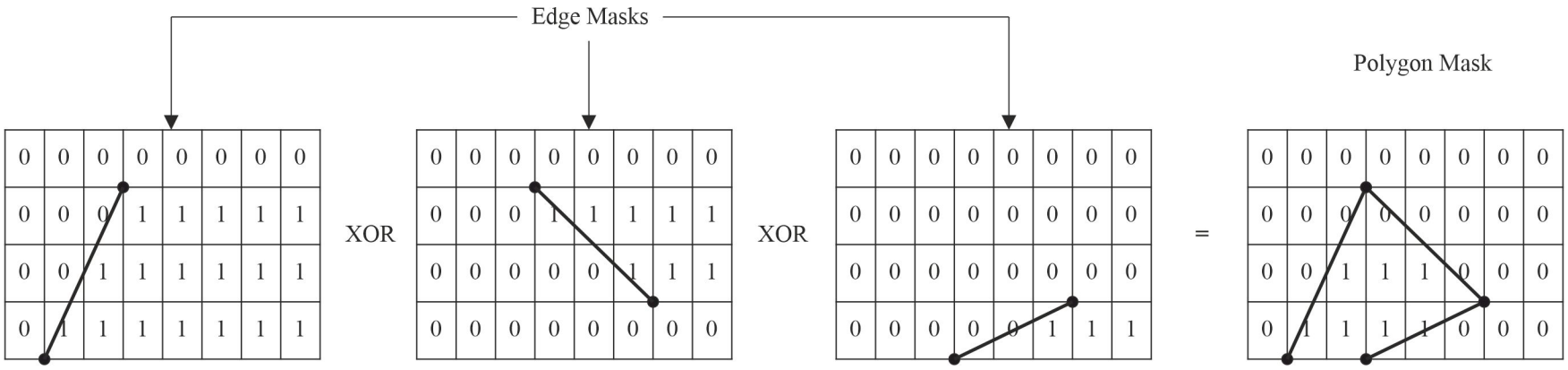
3. Compute final pixel color: $A_0C_0 + A_1C_1 + \ldots + A_{m-1}C_{m-1} + A_BC_B$

   where C_i: the color of polygon I &

   A_B, C_B: background area & its color

Pre-filtering Anti-aliasing: A-buffer

- Catmull's algorithms not practically viable:
  - Extraordinary computations
  - A polygon may not have constant color in a pixel (texture)
- A-buffer is a discrete approximation
  - Use masks and logical operations to approximate pixel coverage
  - Primitives are processed in depth order (front-to-back)
- Computing convex polygon mask from precomputed edge masks (8x4, 32bit)
  - 8x4 represent 9x5 grid, 45^2 edge masks in total



Pre-filtering Anti-aliasing: A-buffer (2)

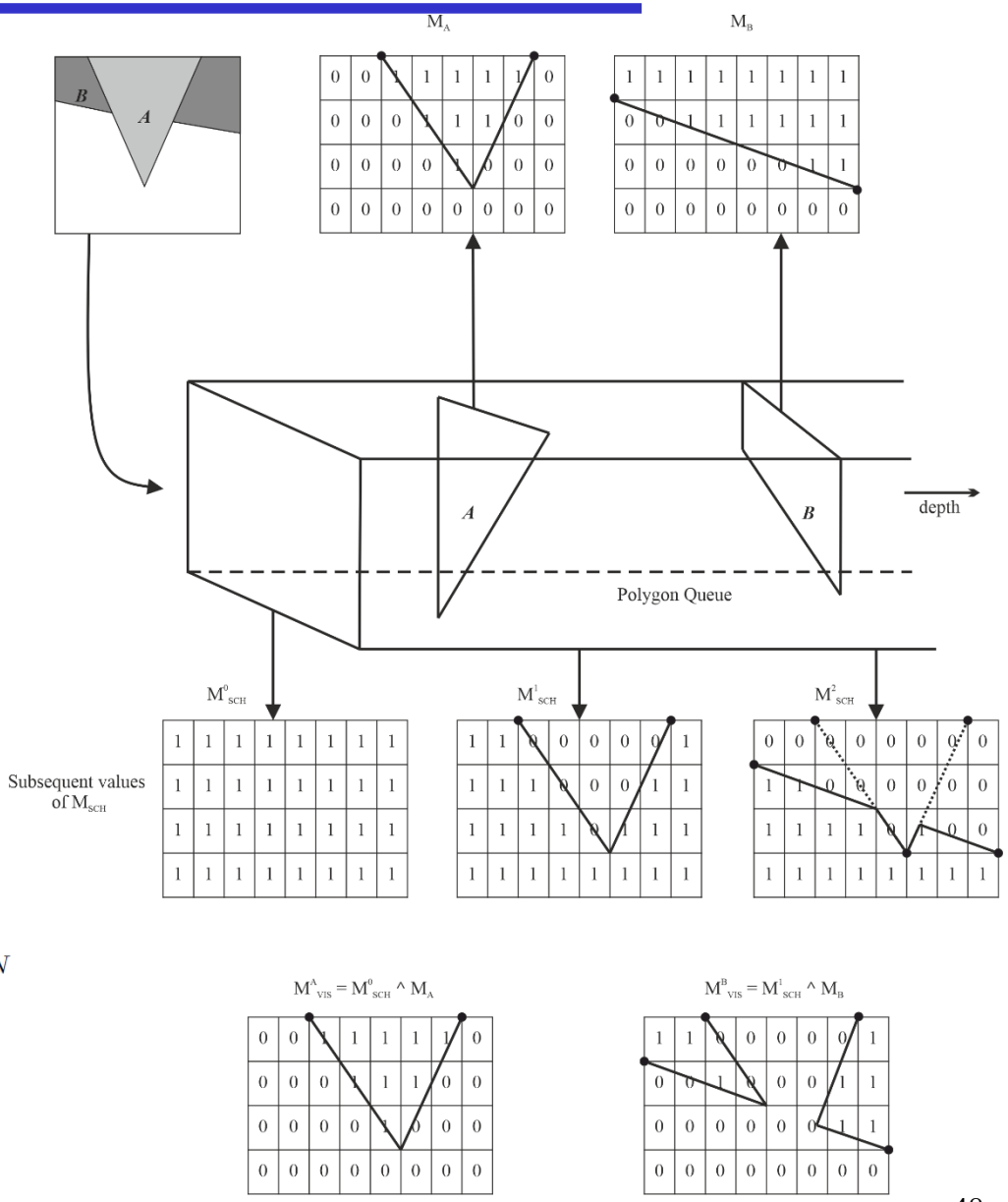
- Polygon mask is used to blend polygon's colour into pixel colour
- M_{SCH}: pixel area not yet covered
  - Initialised to all 1's
  - Updated for each primitive

$$M_{VIS} = M_{SCH} \wedge M_{POL}$$

$$M'_{SCH} = M_{SCH} \wedge \bar{M}_{POL}$$

- #1's in M_{VIS} / 32 gives polygon colour weight A_i

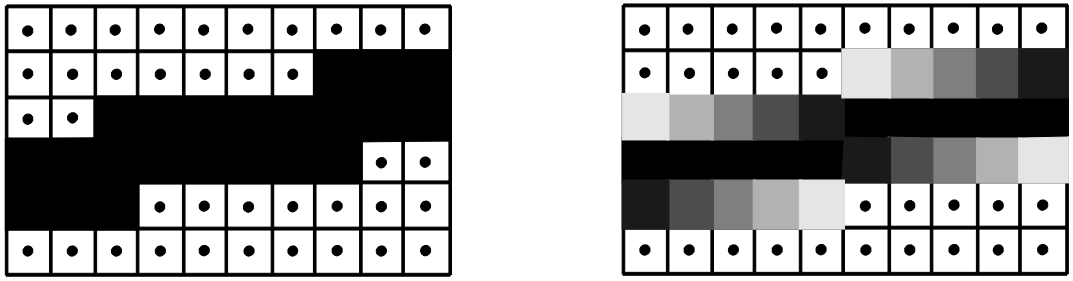
$$C_{pixel} = C_1 * A_1 + C_2 * A_2 + \ldots + C_N * A_N$$



Pre-filtering Anti-aliasing: Line Rasterization

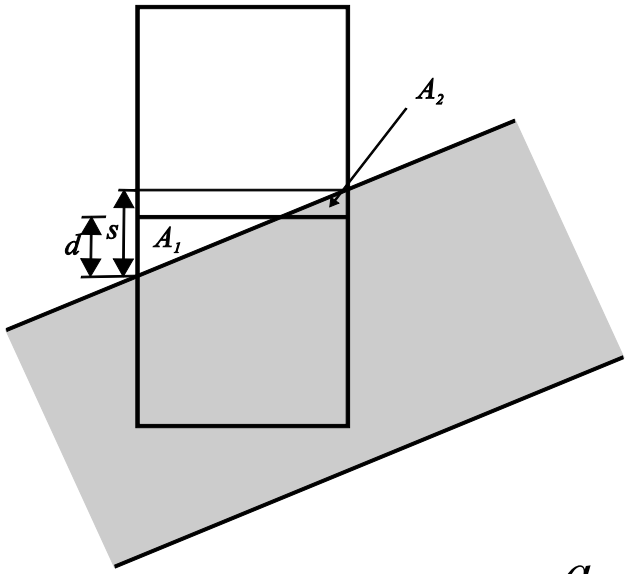
Anti-aliased Line Rasterization

- Bresenham algorithm
  - uses binary decision to select the closest pixel to the mathematical path of the lines → jagged lines & polygon edges
- Lines must have certain width → modeled as thin parallelograms
  - binary decision is wrong
  - color value depends on the % of the pixel that is covered by the line



Pre-filtering Anti-aliasing: Line Rasteriz. (2)

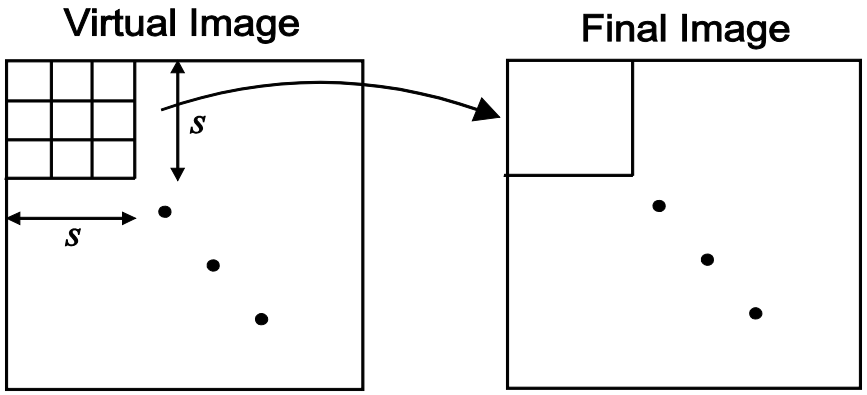
- An example:



- Line in the 1st octant with slope $s = -\frac{a}{b}$
- 2 pixels partially covered by the line
- Determine the portions of the triangles A_1 & A_2
- Color of the top pixel = color of line at a portion A_2
- Color of the bottom pixel = color of line at a portion $(1-A_1)$
- The areas of the triangles: $A_1 = \frac{d^2}{2s}$ $A_2 = \frac{(s-d)^2}{2s}$

Post-filtering Anti-aliasing: SSAA

- More than 1 sample per pixel → image at a higher resolution
- The results are averaged down to the resolution of the pixel grid
- Most common technique due to its simplicity
- An example:
  - to create an 1024 × 1024 image, take 3072 × 3072 samples
    - 9 samples per pixel (3 horizontally × 3 vertically)
  - 3 × 3 virtual image pixels correspond to 1 final image pixel
  - the final pixel's color is the average of the 9 samples



Post-filtering Anti-aliasing: SSAA (2)

- Usually an s x s convolution filter h is used, instead of simply averaging the s x s samples
  - Give more weight to central sample
  - Move filter by s samples in scan-line order
  - The larger the s is → better results, but slower
- Examples of convolution filters for SSAA

1	2	1				1	2	3	2	1				1	2	3	4	3	2	1				
2	4	2				2	4	6	4	2				2	4	6	8	6	4	2				
1	2	1				3	6	9	6	3				3	6	9	12	9	6	3				
						2	4	6	4	2				4	8	12	16	12	8	4				
						1	2	3	2	1				3	6	9	12	9	6	3				
														2	4	6	8	6	4	2				
														1	2	3	4	3	2	1				
3 x 3						5 x 5								7 x 7										

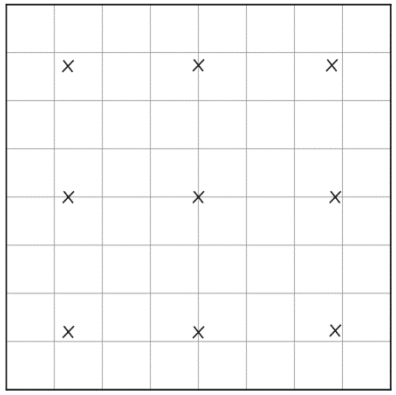
  - To avoid colour shifts, filter must be normalized

$$\sum_{p=0}^{s-1} \sum_{q=0}^{s-1} h(p,q) = 1$$

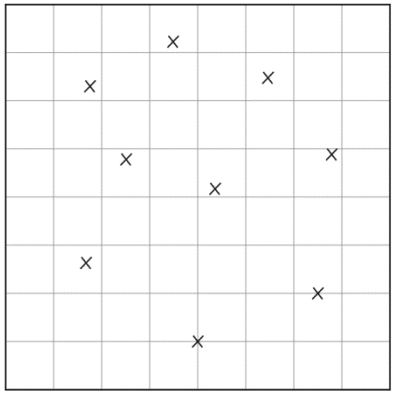
Post-filtering Anti-aliasing: SSAA (3)

- SSAA Drawbacks:
  - ↑s → ↑image generation time & ↑ memory required
  - no matter how big s becomes, the aliasing problem will remain
  - not sensitive to image complexity → a lot of wasted computations

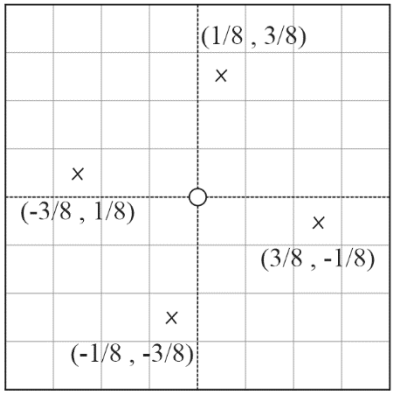
- Sampling pattern is important
  - Regular
  - 4-Rook
    - Good quality (less blur)
  - Stochastic
    - High frequencies become noise
    - Lose incremental computations



Regular



Stohastic

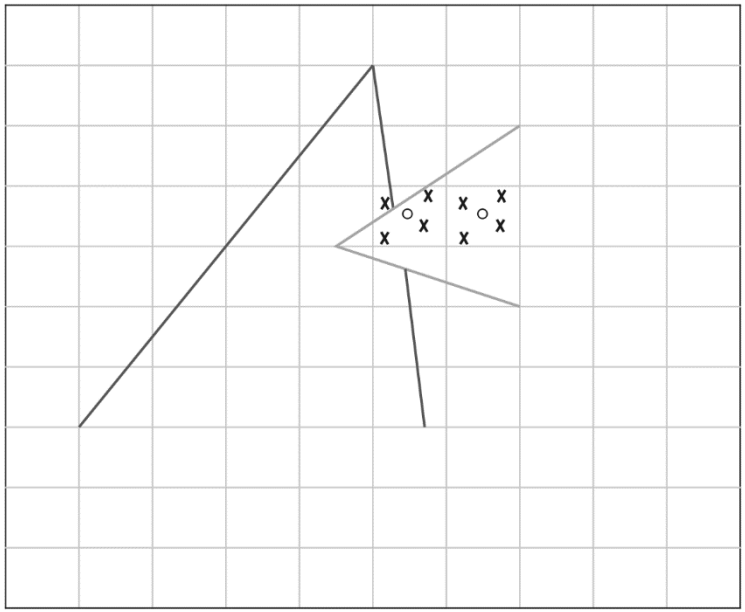


4-Rook

Post-filtering Anti-aliasing: MSAA

- SSAA increases computational cost by s^2
  - Rasterization, HSE, lighting, texturing, transparency… * s^2
  - Much of this is wasted, e.g. in interior areas of objects
- Multi Sample Antialiasing (MSAA) attempts to reduce this cost
  - MSAA performs rasterization and HSE at s^2 resolution
  - *Shading* operations (lighting, texturing, transparency) are performed once per pixel and per fragment
  - The visible portion of a primitive within a pixel (determined at the higher resolution) acts as weight for the shading values (determined at the lower resolution)

Post-filtering Anti-aliasing: MSAA (2)



High resolution sampling positions in 4-rook pattern (x) and low resolution sampling positions (o)

- MSAA disadvantage: does not address aliasing due to shading, e.g. from high resolution textures

Post-processing Anti-aliasing: FXAA

- To drastically reduce the cost of antialiasing for real-time graphics, one has to sample at the display resolution
- Post-processing techniques attempt to detect and remove the effects of aliasing after the image is rasterized

- Fast Approximate Antialiasing (FXAA) detects jaggies on the *intensity image* and blurs them
- 1. the intensity of a pixel $p=(p_r,p_g,p_b)$ can be estimated as

$$I(p) = 0.299p_r + 0.587p_g + 0.114p_b$$

which is sometimes approximated by just taking the green component

Post-processing Anti-aliasing: FXAA (2)

- 2. high contrast pixels (jaggie elements) are detected using a high pass filter, which compares the difference between the *min* and *max* intensities in the von Neumann (N,E,W,S) neighbourhood of *p* against a threshold:

$$I_{range}(p) = max(LN(p)) - min(LN(p)), LN(p) = \{I(p_i) \mid p_i \in N_{vN}(p) \cup p\}$$

- 3. check if high contrast pixels $p=p_{i,j}$ are horizontal or vertical edges using two edge detectors:

$$E_h(p_{i,j}) = \left|\frac{1}{4}I(p_{i-1,j-1}) - \frac{1}{2}I(p_{i,j-1}) + \frac{1}{4}I(p_{i+1,j-1})\right| + \left|\frac{1}{2}I(p_{i-1,j}) - I(p_{i,j}) + \frac{1}{2}I(p_{i+1,j})\right| + \left|\frac{1}{4}I(p_{i-1,j+1}) - \frac{1}{2}I(p_{i,j+1}) + \frac{1}{4}I(pi+1,j+1)\right|$$

$$E_v(p_{i,j}) = \left|\frac{1}{4}I(p_{i-1,j-1}) - \frac{1}{2}I(p_{i-1,j}) + \frac{1}{4}I(p_{i-1,j+1})\right| + \left|\frac{1}{2}I(p_{i,j-1}) - I(p_{i,j}) + \frac{1}{2}I(p_{i,j+q})\right| + \left|\frac{1}{4}I(p_{i+1,j-1}) - \frac{1}{2}I(p_{i+1,j}) + \frac{1}{4}I(p_{i+1,j+1})\right|$$

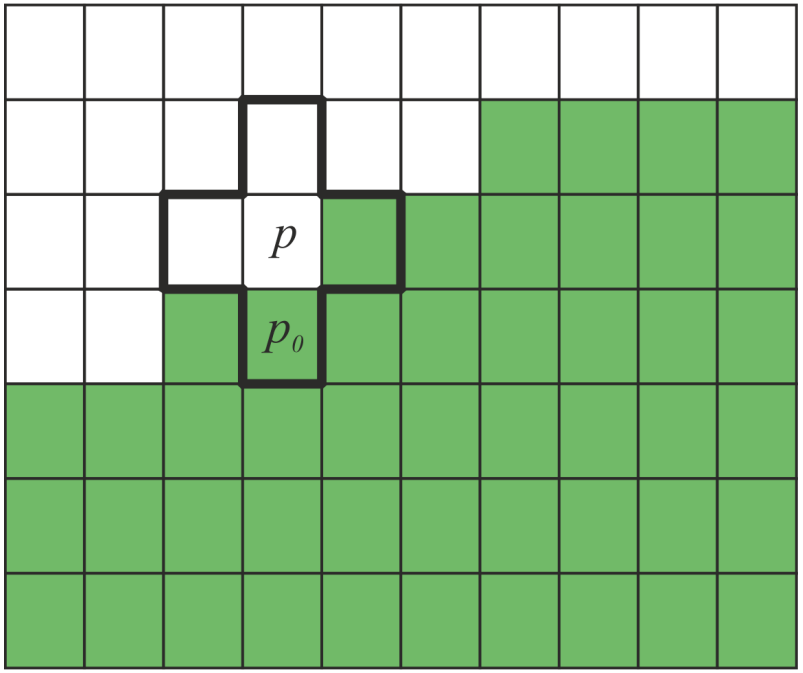
If $E_h > E_v$ then we have a horizontal edge, else a vertical one

Post-processing Anti-aliasing: FXAA (3)

- 4. select the highest contrast pixel $p_o \in N_{vN}$ to be blended with $p=p_{i,j}$:

$$\text{if } E_h > E_v \text{ then } p_o = (|p_{i,j} - p_{i-1,j}| > |p_{i,j} - p_{i+1,j}|)?p_{i-1,j} : p_{i+1,j}$$

$$\text{else if } E_h \leq E_v \text{ then } p_o = (|p_{i,j} - p_{i,j-1}| > |p_{i,j} - p_{i,j+1}|)?p_{i,j-1} : p_{i,j+1}$$



Post-processing Anti-aliasing: FXAA (4)

- 5. estimate the blend factor α to blur p and p_o as $p' = (1-\alpha)p + \alpha p_o$
  - α depensds on how much p stands out in its neighbourhood
  - calculated as the normalized difference between p and the average intensity of its 8-neighbourhood

$$\alpha = \frac{|I(p_{i,j}) - I_{avg}(p_{i,j})|}{I_{range}(p_{i,j})}$$

$$I_{avg}(p_{i,j}) = \frac{1}{12}\begin{bmatrix}1 & 2 & 1\end{bmatrix}\begin{bmatrix}I(p_{i-1,j-1}) & I(p_{i,j-1}) & I(p_{i+1,j-1}) \\ I(p_{i-1,j}) & 0 & I(p_{i+1,j}) \\ I(p_{i-1,j+1}) & I(p_{i,j+1}) & I(p_{i+1,j+1})\end{bmatrix}\begin{bmatrix}1 \\ 2 \\ 1\end{bmatrix}$$

Post-processing Anti-aliasing: DLAA/DLSS

- Why not teach a Neural Network (NN) to antialias?
  - Training set:
    - High quality images without aliasing (the Ground Truth)
    - Synthetically generated aliased versions of them (the Input)
  - Can use a supercomputer to train off-line
  - Pioneered by Nvidia
  - Two options:
    - Deep Learning Antialiasing (DLAA): objective to improve image quality. Render image and apply NN.
    - Deep Learning Super Sampling (DLSS): objective to increase rendering performance. Render at reduced resolution, upsample with bilinear interpolation (aliasing!), apply NN on upsampled image.
  - DLAA is better at a cost

Post-processing Anti-aliasing: DLAA/DLSS (2)

- DLSS

2D Clipping Algorithms

- Avoid giving out-of-range values to a display device
- **Clipping object (window)**: display device usually modeled as rectangular parallelogram which defines the within-range values
- **Subject**: primitive of a modeled scene
- Generalization from 2D to 3D is relatively straightforward
- Subject relation to the clipping object
  - Subject entirely inside: rasterize it
  - Subject outside: do not rasterize
  - Subject intersects the clipping object: compute the intersection with a 2D clipping algorithm & rasterize the result

Point Clipping

- Point clipping is a trivial case:
  - is point (x, y) inside the clipping object ?
- If the clipping object is a rectangular parallelogram:
  - Exploit its opposite vertices (x_{min}, y_{min}), (x_{max}, y_{max})
- Inclusion Test:

  If $x_{min} \leq x \leq x_{max}$ & $y_{min} \leq y \leq y_{max}$

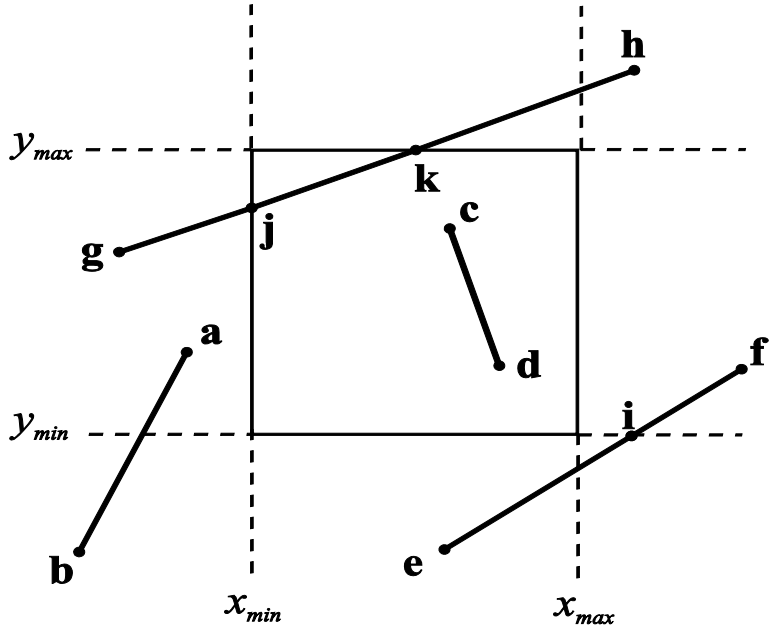
  Then the point is entirely inside and must be rasterized

  Else the point is entirely outside and must NOT be rasterized

Line Clipping - CS Algorithm

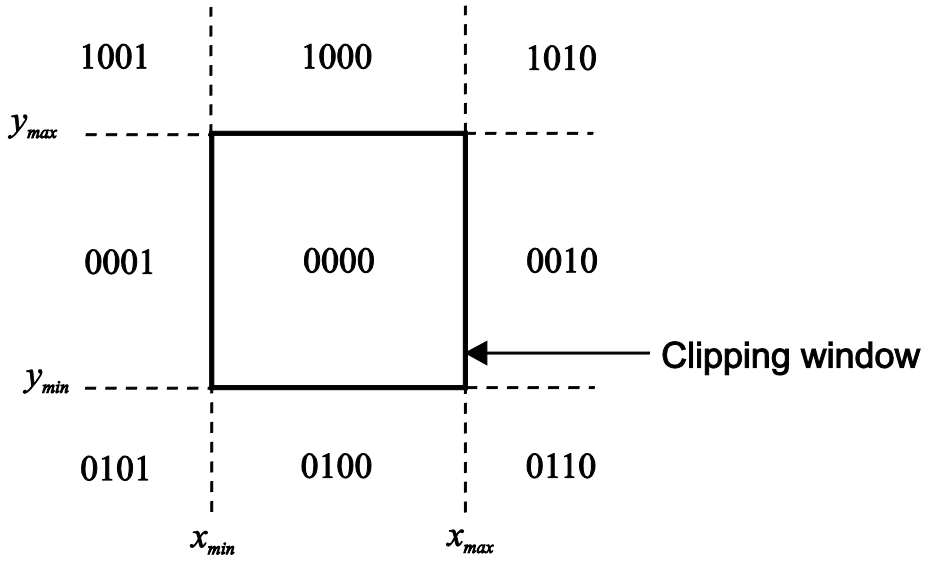
Cohen – Sutherland (CS) Algorithm

- Perform a low-cost test which decides if a line segment is entirely inside or entirely outside the clipping window
- For each non-trivial line segment compute its intersection with one of the lines defined by the window boundary
- Recursively apply the algorithm to both resultant line segments



Line Clipping - CS Algorithm (2)

- The plane of the clipping window is divided into 9 regions
- Each region is assigned a 4 – bit binary code
- The code bits are set according to the following rules:
  - **First Bit**: Set 1 for $y > y_{max}$, else set 0
  - **Second Bit**: Set 1 for $y < y_{min}$, else set 0
  - **Third Bit**: Set 1 for $x > x_{max}$, else set 0
  - **Fourth Bit**: Set 1 for $x < x_{min}$, else set 0



Line Clipping - CS Algorithm (3)

- Let the 4 – bit codes of the endpoints of a line segment be c_1, c_2
- Each endpoint is assigned a 4 – bit code according to the above rules
- Then the low-cost inclusion tests are:
  - If $c1 \vee c2 = 0000$

    Then the line segment is entirely inside

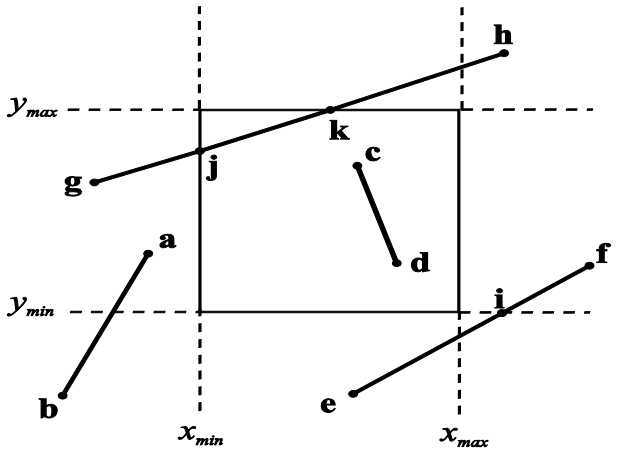
  - If $c1 \wedge c2 \neq 0000$

    Then the line segment is entirely outside

Line Clipping - CS Algorithm (4)

- Example :

Endpoint	Code	Endpoint	Code
a	0001	e	0100
b	0101	f	0010
c	0000	g	0001
d	0000	h	1010



- **ab** is entirely outside since $0001 \wedge 0101 \neq 0000$
- **cd** is entirely inside since $0000 \vee 0000 = 0000$
- For **ef** & **gh** the extent tests are not conclusive ⇨ compute the intersection points
- Intersect **ef** with line $y = y_{min}$ since the 2nd bit of the code is different at **e** & **f**
- Continue with the **if** line segment as the 2nd bit of the code of the **f** vertex has value 0 (inside)
- For **gh** compute one of the intersection points **k** & continue with **gk** which then computes the intersection **j** & recurses with a trivial inside decision for **jk**

Line Clipping – LB Algorithm

Liang – Barsky (LB) Algorithm

- Solves the line clipping problem without using recursive calls
- Compared to CS algorithm, LB is more than 30% more efficient
- Can be easily extended to a 3D clipping object
- LB is based on the parametric equation of the line segment to be clipped from $\mathbf{p_1}(x_1, y_1)$ to $\mathbf{p_2}(x_2, y_2)$:

$$\mathbf{P} = \mathbf{p_1} + t\,(\mathbf{p_2} - \mathbf{p_1}), \quad t \in [0, 1]$$

or

$$x = x_1 + t\,\Delta x\,, \quad y = y_1 + t\,\Delta y$$

where

$$\Delta x = x_2 - x_1\,, \Delta y = y_2 - y_1$$

Line Clipping – LB Algorithm (2)

- For the part of the line segment that is inside the clipping window:

$$x_{min} \le x_1 + t\,\Delta x \le x_{max}\,,$$
$$y_{min} \le y_1 + t\,\Delta y \le y_{max}$$

or

$$-t\,\Delta x \le x_1 - x_{min}\,,$$
$$t\,\Delta x \le x_{max} - x_1\,,$$
$$-t\,\Delta y \le y_1 - y_{min}\,,$$
$$t\,\Delta y \le y_{max} - y_1$$

Line Clipping – LB Algorithm (3)

- The above inequalities have the common form:

$$t\,p_i \le q_i \ ,$$

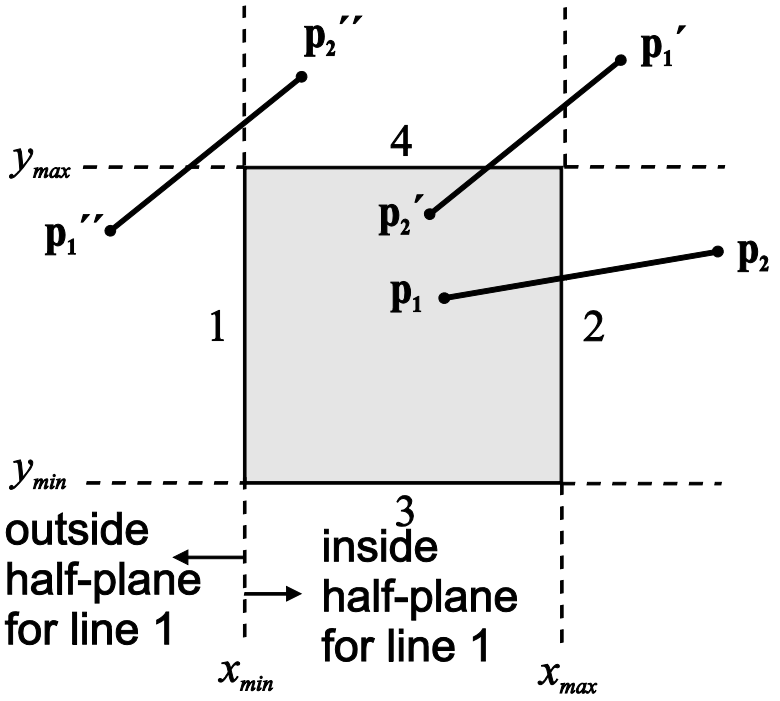
where

$$p_1 = -\Delta x \ , \ q_1 = x_1 - x_{min}$$

$$p_2 = \Delta x \ , \ q_2 = x_{max} - x_1$$

$$p_3 = -\Delta y \ , \ q_3 = y_1 - y_{min}$$

$$p_4 = \Delta y \ , \ q_4 = y_{max} - y_1$$



Line Clipping – LB Algorithm (4)

- Notice the following:
  - If $p_i = 0$ the line segment is parallel to the window edge i and the clipping problem is trivial
  - If $p_i \neq 0$ the parametric value of the point of intersection of the line segment with the line defined by window edge i is

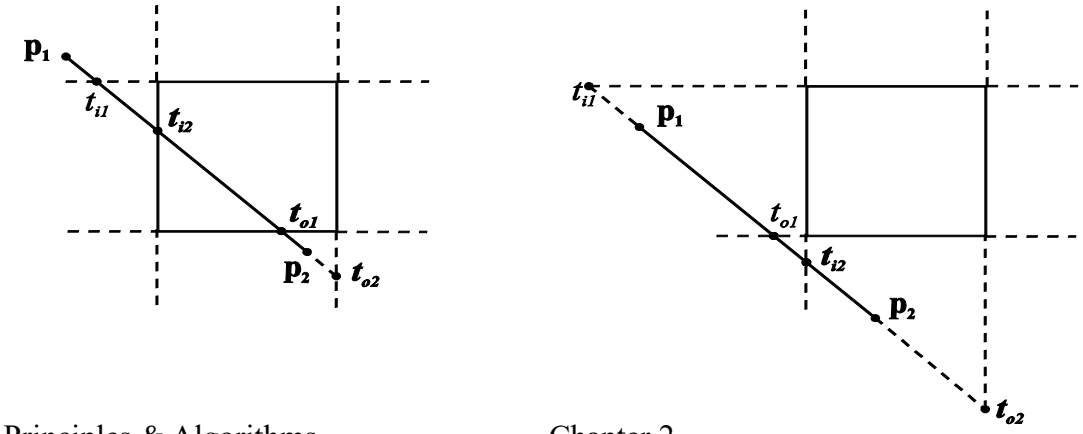
    $t_i = q_i / p_i$
  - If $p_i < 0$ the directed line segment is incoming with respect to window edge i
  - If $p_i > 0$ the directed line segment is outgoing with respect to window edge i

Line Clipping – LB Algorithm (5)

- Therefore t_{in} and t_{out} can be computed as:

$$t_{in} = \max(\{\frac{q_i}{p_i} \mid p_i < 0,\ i:1..4\} \cup \{0\}),\quad t_{out} = \min(\{\frac{q_i}{p_i} \mid p_i > 0,\ i:1..4\} \cup \{1\})$$

- Sets $\{0\}$, $\{1\}$ clamp the starting and ending parametric values at the end points of the line segment
- If $t_{in} \le t_{out}$, the values t_{in} and t_{out} are plugged into parametric line equation to get the actual starting – ending points of the clipped segment
- Otherwise there is no intersection with the clipping window



Line Clipping – LB Algorithm (6)

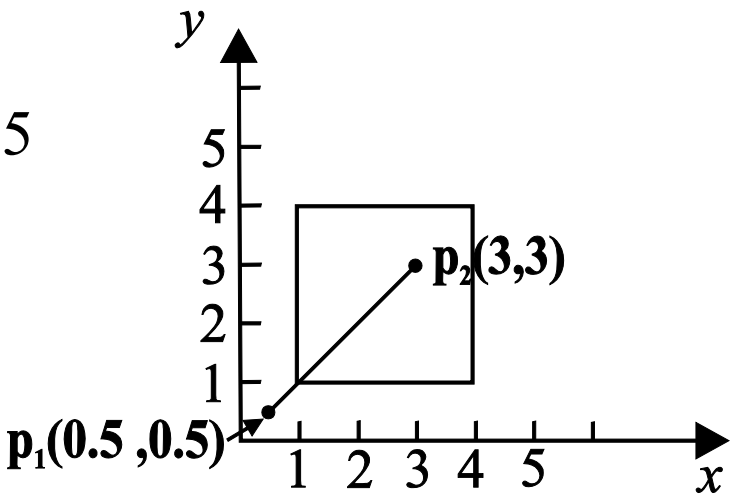
LB example:

- Compute: $\Delta x = 2.5$ and $\Delta y = 2.5$
- Compute: $p_1 = -2.5,\ q_1 = -0.5$

  $p_2 = 2.5,\ q_2 = 3.5$

  $p_3 = -2.5,\ q_3 = -0.5$

  $p_4 = 2.5,\ q_4 = 3.5.$

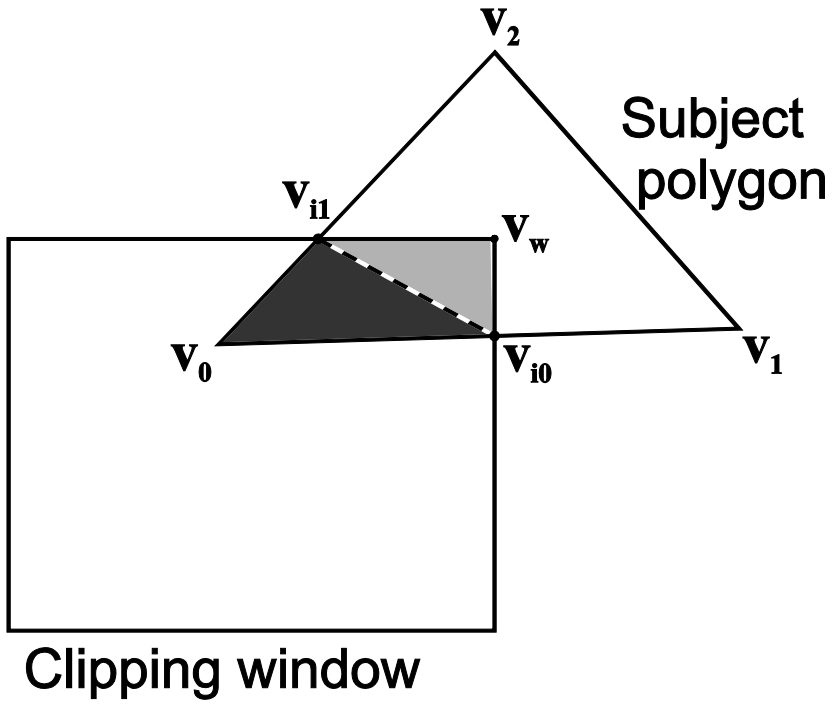


- Compute: $t_{in} = \max(\{\frac{q_1}{p_1}, \frac{q_3}{p_3}\} \cup \{0\}) = 0.2$, $t_{out} = \min(\{\frac{q_2}{p_2}, \frac{q_4}{p_4}\} \cup \{1\}) = 1$
- Since $\boldsymbol{t_{in} < t_{out}}$ compute endpoints $\mathbf{p_1}'(x_1', y_1')$, $\mathbf{p_2}'(x_2', y_2')$ of the clipped line segment using the parametric equation:

$$x_1' = x_1 + t_{in}\Delta x = 0.5 + 0.2 \cdot 2.5 = 1$$
$$y_1' = y_1 + t_{in}\Delta y = 0.5 + 0.2 \cdot 2.5 = 1$$
$$x_2' = x_1 + t_{out}\Delta x = 0.5 + 1 \cdot 2.5 = 3$$
$$y_2' = y_1 + t_{out}\Delta y = 0.5 + 1 \cdot 2.5 = 3$$

Polygon Clipping

- In 2D polygon clipping the subject and clipping object are both polygons (*subject polygon, clipping polygon*)
- Why is polygon clipping important ?

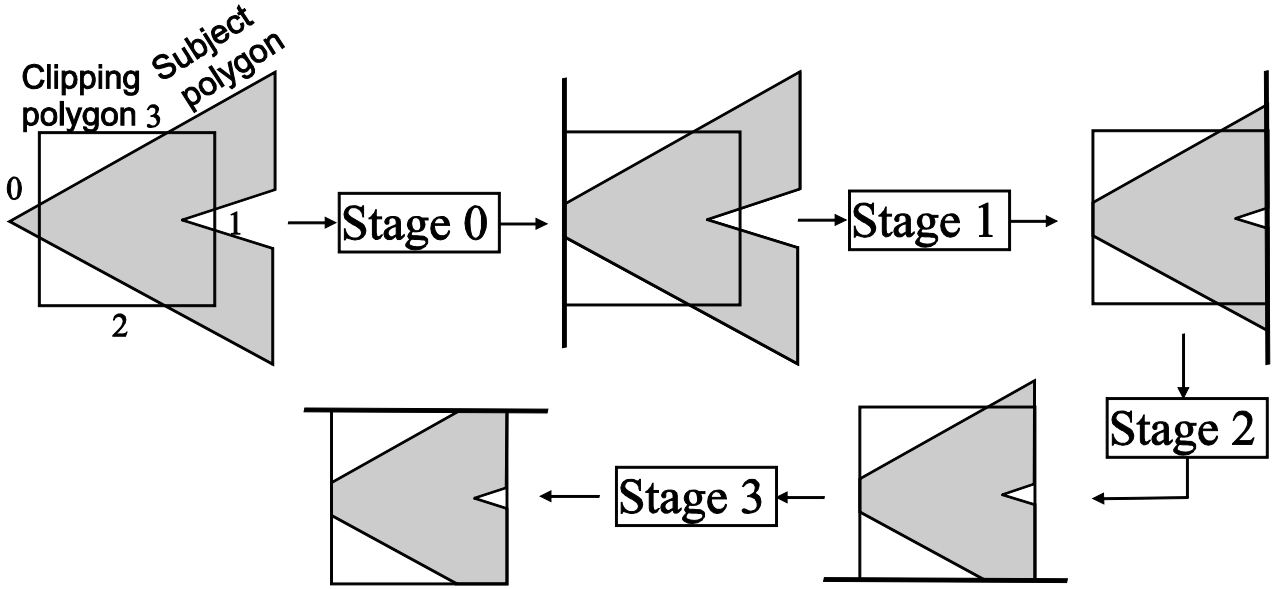


- Polygon clipping cannot be regarded as multiple line clipping

Polygon Clipping – SH Algorithm

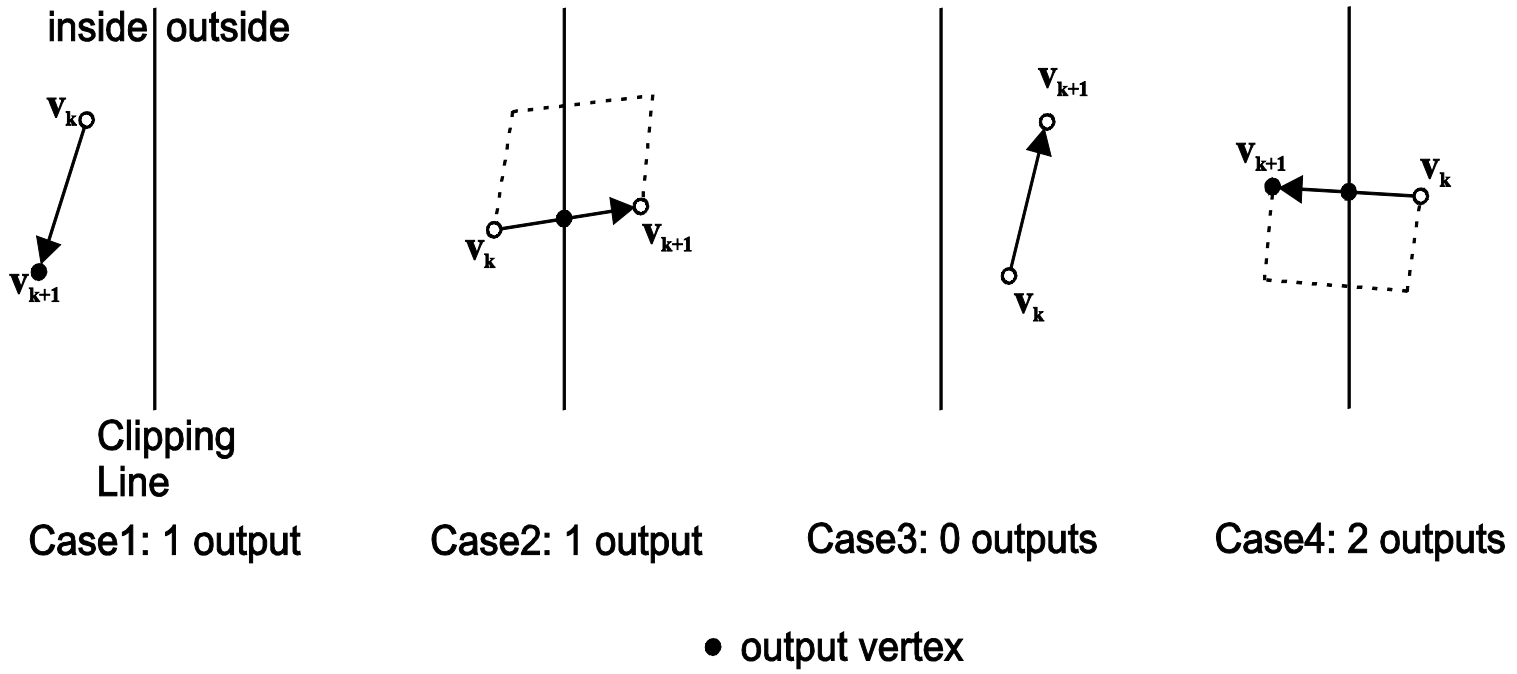
Sutherland – Hodgman (SH) Algorithm:

- Clips an arbitrary subject polygon against a **convex** clipping polygon
- Has *m* pipeline stages which correspond to the *m* edges of the clipping polygon
- Stage i | i: 0…*m*-1 clips the subject polygon against the line defined by edge i of the clipping polygon
- The input to stage i | i: 1…*m*-1 is the output of stage i-1
- Polygon is restricted to be convex



Polygon Clipping – SH Algorithm (2)

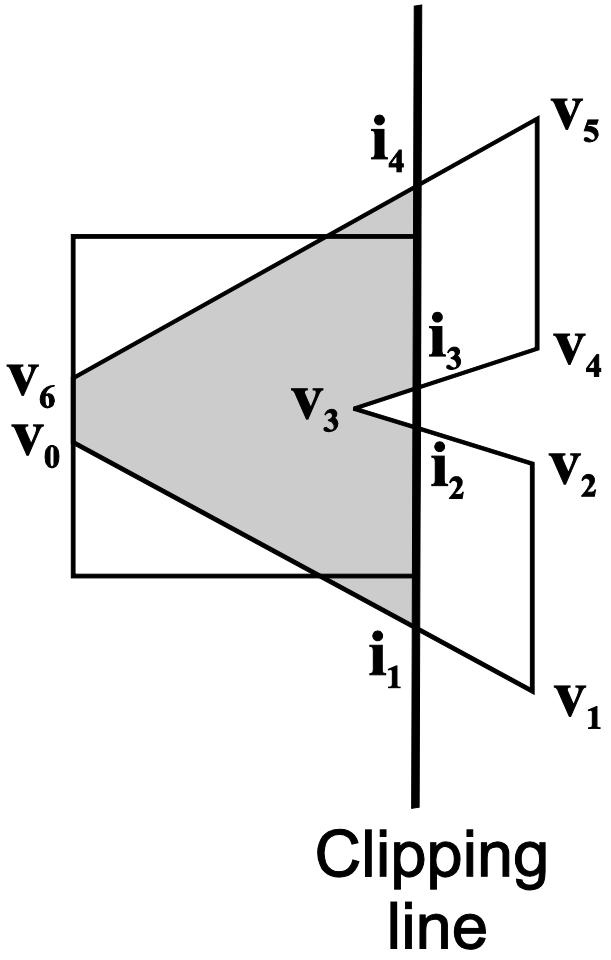
- For each stage of the SH algorithm there are the following 4 relationships between a clipping line and an object polygon edge $\mathbf{v_k v_{k+1}}$



Polygon Clipping – SH Algorithm (3)

- Example of the 1st stage of the SH algorithm:

v_k	v_{k+1}	Case	Output
v_0	v_1	2	i_1
v_1	v_2	3	-
v_2	v_3	4	i_2, v_3
v_3	v_4	2	i_3
v_4	v_5	3	-
v_5	v_6	4	i_4, v_6
v_6	v_0	1	v_0



Polygon Clipping – SH Algorithm (4)

- *Algorithm:*

```
polygon SH_Clip ( polygon C, S ) { /*C must be convex*/
   int i, m;
   edge e;
   polygon InPoly, OutPoly;
   m = getedgenumber(C);
   InPoly = S;
   for (i=0; i<m; i++) {
        e = getedge(C,i);
        SH_Clip_Edge(e,InPoly,OutPoly);
        InPoly = OutPoly
        }
   return OutPoly
}
```

Polygon Clipping – SH Algorithm (5)

- *Algorithm:*

```
SH_Clip_Edge ( edge e, polygon InPoly, OutPoly ) {
    int k, n; vertex vk, vkplus1, i;
    n = getedgenumber(InPoly);
    for (k=0; k<n; k++) {
        vk = getvertex(InPoly,k); vkplus1=getvertex(InPoly,(k+1) mod n);
        if (inside(e, vk) and inside(e, vkplus1))
                /* Case 1 */
                putvertex(OutPoly,vkplus1)
        else if (inside(e, vk) and !inside(e, vkplus1)) {
                /* Case 2 */
                i = intersect_lines(e, (vk,vkplus1)); putvertex(OutPoly,i)
        }
        else if (!inside(e, vk) and !inside(e, vkplus1))
                /* Case 3 */
        else {
                /* Case 4 */
                i = intersect_lines(e, (vk,vkplus1)); putvertex(OutPoly,i);
                putvertex(OutPoly,vkplus1)
        }
    }
}
```

Polygon Clipping – SH Algorithm (6)

- The complexity of SH algorithm is *O(mn)* where *m* and *n* are the numbers of vertices of the clipping and subject polygons respectively

- No complex data structures or operations are required so the SH algorithm is quite efficient

- The SH algorithm is appropriate for hardware implementation since the clipping polygon, in general, is constant

Polygon Clipping – GH Algorithm

Greiner – Hormann Algorithm

- Suitable for general clipping polygons (*C*) and subject polygons (*S*)
- The polygons can be arbitrary closed polygons, even self intersecting
- The complexity of steps 1 and 2 is *O(mn)* where *m* and *n* are the numbers of vertices of the *C* and *S* polygons respectively
- The overall complexity of the GH algorithm is *O(mn)*
- In practice, the complex data structures used in the GH algorithm make it less efficient than SH

Polygon Clipping – GH Algorithm (2)

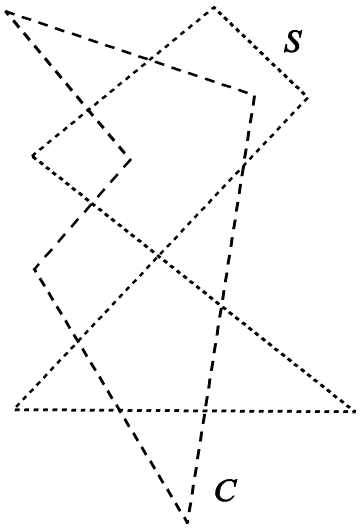
- GH algorithm is based on the winding number test for point **p** in polygon P, symbolically → $\omega(P, \mathbf{p})$
- $\omega(P, \mathbf{p})$ does not change so long as the topological relation of the point **p** and the polygon P remains constant
- If p crosses P the $\omega(P, \mathbf{p})$ is incremented or decremented
- If $\omega(P, \mathbf{p})$ is odd then **p** is inside P, otherwise it is outside

Polygon Clipping – GH Algorithm (3)

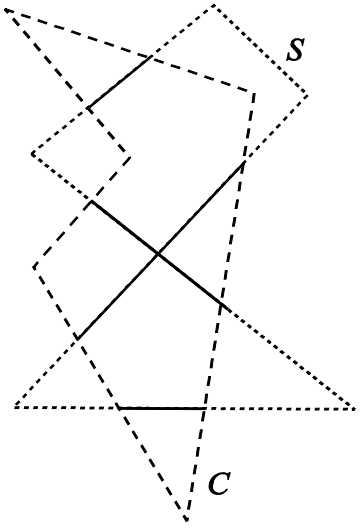
- The 3 steps of the GH algorithm:
  1. Trace the perimeter *S* starting from a vertex $\mathbf{v_{s0}}$. An imaginary stencil toggles between *on* and *off* state every time the perimeter of *C* is crossed. Its initial state is *on* if $\mathbf{v_{s0}}$ is inside *C* and *off* otherwise. It thus computes the part of the perimeter of *S* that is inside *C*
  2. As step 1 but reverse the roles of *S* and *C*. The part of the perimeter of *C* that is inside *S* is thus computed
  3. The union of the results of steps 1 and 2 is the result of clipping *S* against *C* (or equivalently *C* against *S*)

Polygon Clipping – GH Algorithm (4)

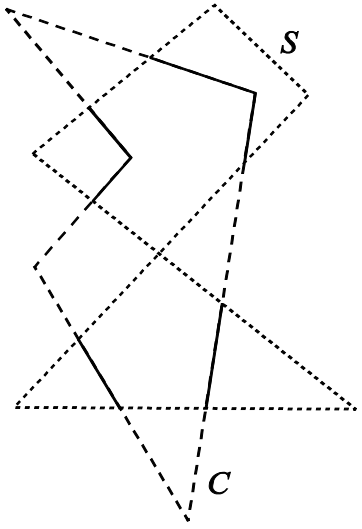
- GH algorithm example:
  - (a) The initial *S*, *C* polygons
  - (b) After step 1 of GH
  - (c) After step 2 of GH
  - (d) The final result



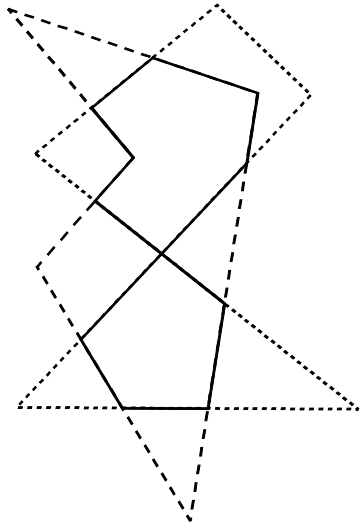
(a)



(b)



(c)


(d)

Polygon Clipping – GH Algorithm (5)

- GH algorithm computes the intersection of the areas of 2 polygons, $C \bigcap S$
- It easily generalizes to compute $C \bigcup S$, C–S and S–C by changing the initial states of the stencils for S and C
- Obviously there are 4 possible combinations of the initial state
- These generalizations are not useful for the clipping problem